Lab 4a - Introduction to Perceptrons

In the first part of this lab, we will learn about gradient desecent, an optimization algorithm. We will also build a perceptron from scratch and show how we can use gradient descent to find the weights.

Part B of the lab will apply this in a neural network for regeression to predict COVID19 cases.

Created by: Tuan Do

In [ ]:
```
## Code snippet to wrap the lines when converting to PDFs
from IPython.display import HTML, display

display(HTML("""
<style>
pre, code {
  white-space: pre-wrap !important;
  word-wrap: break-word !important;
  overflow-wrap: anywhere !important;
}
.jp-RenderedHTMLCommon pre {
  white-space: pre-wrap !important;
}
</style>
"""))
```

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
import sklearn as sk
from sklearn.metrics import accuracy_score
from sklearn import datasets, model_selection
import pandas as pd
import seaborn as sns # reminder: you may use any plotting tool you like!
```

Gradient Descent

We will introduce gradient descent by fitting a line, also called linear regression. You wouldn't ordinarily use gradient descent with linear regression because you can just solve directly with the closed form least squares equation. But, for demonstration purposes we will use gradient descent on a linear regression example. In this way, using gradient descent which is iterative, we can tweak various hyperparameters to minimize a cost function. Hyperparameters include the size of the steps taken to find the minimum, called the learning rate. If the learning rate is too small, the algorithm will have to iterate many times before converging, which is slow. Conversely, if the learning rate is too high,

you might end up not converging to the global minimum. When using gradient descent, you should ensure features have a similar scale to speed up convergence. Oftentimes in ML literature, you will see the learning rate set to a default parameter. Of course, you can test out various learning rates and see what happens, part of our exercise task.

There are a variety of gradient descent algorithms, and the 'best' change often, we will use the batch gradient descent which you would likely not use in real life, but it provides an understanding of the algorithm's inner workings. We will use a common lost function (related to the MSE):

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2$$

(when you see it in this form: $h_\theta = \theta^T x$).

and taking the partial derivatives of the cost function,

$$\frac{\partial}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)}) \cdot x_j^{(i)}$$

This cost function is 'nice' because their first derivatives are convex.
HOML uses the matrix form of the cost function and its partial derivatives (shown here: https://rpubs.com/dnuttle/ml-linear-cost-func-derivative), but here we will break it down into a simpler form so you can see the steps easier. We use the classic y = mx + b function for a line, and loss is the error in the predicted value of m and b (ordinary least squares). We will initialize m and b to 0, and they will be updated iteratively as the error becomes smaller. so then our cost function now becomes

$$E = \frac{1}{n} \sum_{i=1}^{n} (y_i - (mx_i + b))^2$$

Notice we changed m to n because m is now our slope. And what happened to the 1/2 in the other cost function? Well, we are showing you what you'll see out there in the real world- some are multiplied by 1/2 so when you take the partial derivatives they cancel, at any rate its just a constant so it doesn't matter for optimization. Now, we need the partial derivatives with respect to m and b:

$$D_m = \frac{-2}{n} \sum_{i=1}^{n} x_i (y_i - \hat{y}_i)$$

which becomes in our code: `D_m = (-2/n) * sum(X * (y - Y_pred))` # Derivative wrt m.

Likewise the partial derivative wrt b is:

$$D_b = \frac{-2}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)$$

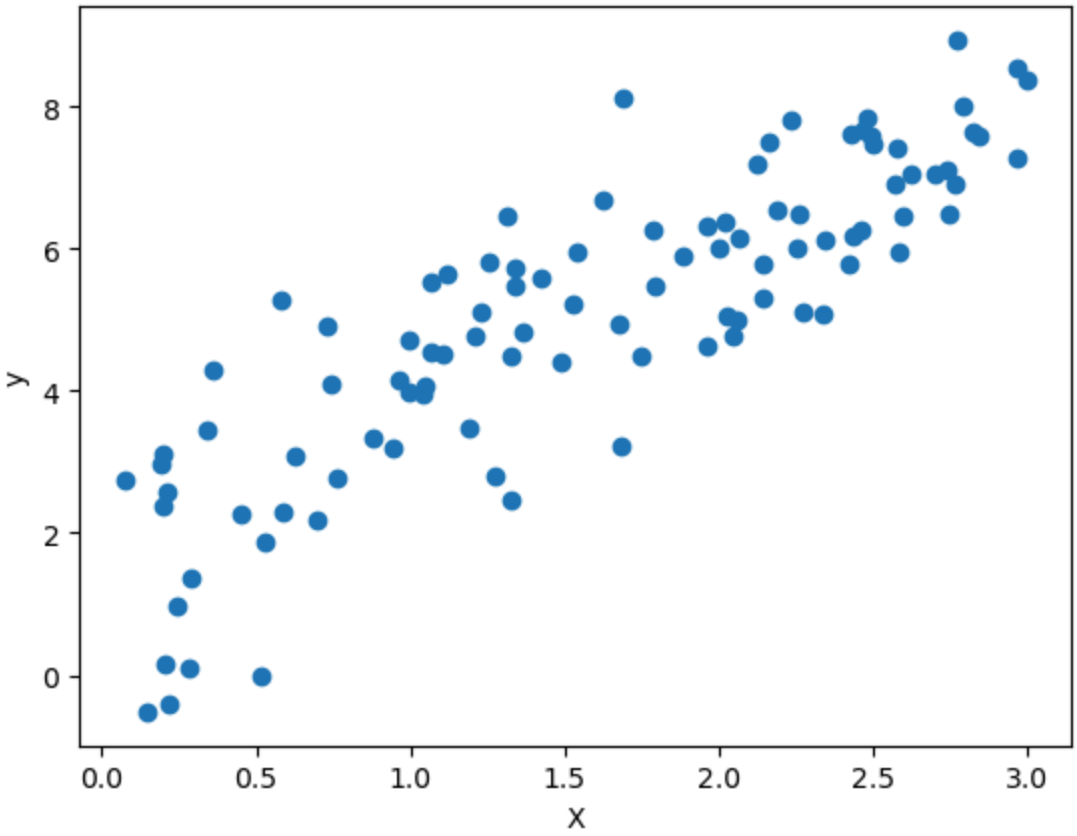
(see https://ml-cheatsheet.readthedocs.io/en/latest/linear_regression.html for how to compute each partial derivative) I'll leave it to you as an exercise to translate into code.

In [2]:
```
## Create some linear data
# Let's  generate some random linearish data so we can fit it with gradient
descent

X = 3* np.random.rand(100,1)
y = 2 + 2*X + np.random.randn(100,1)

plt.scatter(X,y)
plt.xlabel('X')
plt.ylabel('y')
```

Out[2]: Text(0, 0.5, 'y')



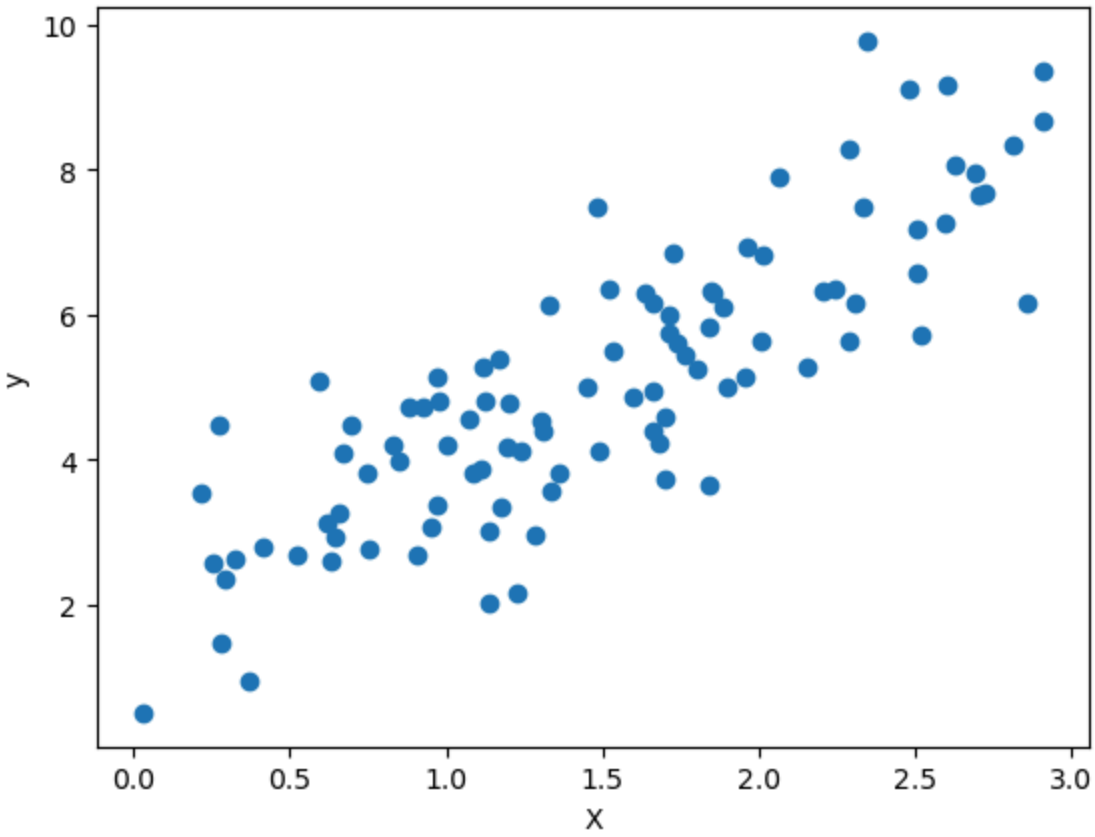
In [3]:
```
## Create some linear data
# Let's  generate some random linearish data so we can fit it with gradient
descent

X = 3* np.random.rand(100,1)
y = 2 + 2*X + np.random.randn(100,1)

plt.scatter(X,y)
```

```
plt.xlabel('X')
plt.ylabel('y')
```

Out[3]: Text(0, 0.5, 'y')



Question 1 (8 pts)

To create a gradient step algorithm, we set m and b to 0 (this starts our prediction where?)
We also need to set a learning rate, let's try L = 0.0001 In our loop,

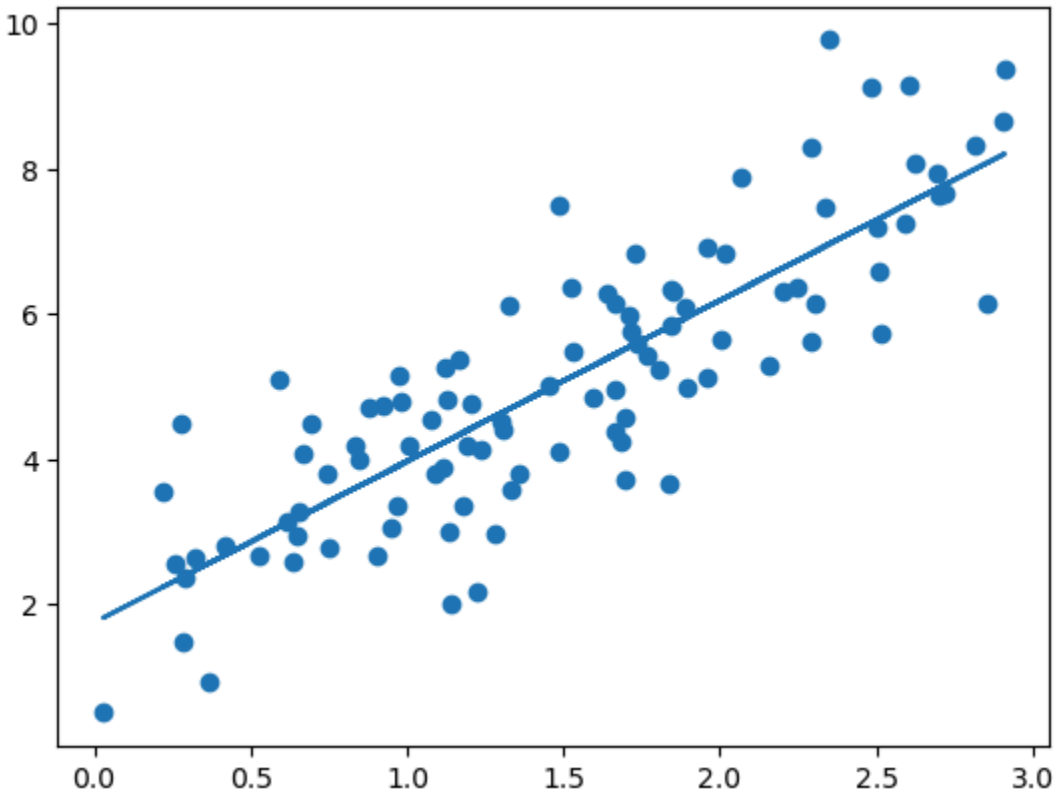
1. set Y_pred = m*X + b, the current predicted value of Y
2. get derivative wrt m, set it to D_m
3. get derivative wrt b, set it to D_b
4. update m by m - L*D_m
5. update b
6. print out m, b and in a cell below plot your best fit line on the scatter plot.
7. Describe the quality of the fit and how it changes as you change the learning rate and number of epochs.
8. Make a plot of the loss as a function of epoch for your best model fit.

In [4]:
```
m = 0
b = 0
L = 0.01  # The learning Rate
epochs = 3000  # The number of iterations to perform gradient descent
n = float(len(X)) # Number of elements in X
def gradient_descent(m,X,y,b):
    n=float(len(X))
    Y_pred=m*X+b
    D_m = (-2/n)*np.dot(X,(y-Y_pred))
    D_b = (-2/n)*(np.sum(y-Y_pred))
    m = m - L*D_m
    b= b-D_b*L
    return m,b
# Performing Gradient Descent
for i in range(epochs):
    # FILL IN CODE HERE
    m,b=gradient_descent(m,X.flatten(),y.flatten(),b)
    # REMOVE THIS

print (m,b)
```

```
2.2201220741983487 1.7470291391164816
```

In [5]:
```
plt.scatter(X,y)
plt.plot(X,m*X+b)
plt.show()
```



Understanding Perceptrons: The Building Blocks of Neural Networks

What is a Perceptron?

A perceptron is one of the simplest forms of artificial neural networks. Invented in 1957 by Frank Rosenblatt, it serves as a binary classifier that makes decisions by combining input signals with weights and applying an activation function.

How Does a Perceptron Work?

A perceptron works in three main steps:

1. **Input Processing**:
   - Takes multiple input signals (x_1, x_2, ..., x_n)
   - Each input has an associated weight (w_1, w_2, ..., w_n)
   - There's also a bias term (b) that helps shift the decision boundary
2. **Weighted Sum**:
   - Multiplies each input by its weight: $w_1x_1 + w_2x_2 + ... + w_nx_n$
   - Adds the bias: $z = w_1x_1 + w_2x_2 + ... + w_nx_n + b$
3. **Activation Function**:
   - Applies an activation function to the weighted sum
   - For binary classification, often uses a step function or sigmoid:
     - Step: output = 1 if $z \geq 0$, else 0
     - Sigmoid: output = 1/(1 + e^(-z))

Training a Perceptron

The perceptron learns through a simple algorithm:

1. Initialize weights and bias randomly
2. For each training example:
   - Make a prediction
   - Compare with actual target value
   - Update weights if prediction is wrong:
     - w_new = w_old + α(target - prediction)x where α is the learning rate

Limitations

Perceptrons have some important limitations:

- Can only learn linearly separable patterns
- Cannot solve XOR problem
- Binary classification only (in its basic form)

Applications

Despite limitations, perceptrons are useful for:

- Understanding basic neural network concepts
- Simple binary classification tasks
- Building blocks for more complex neural networks

Understanding Gradient Descent in the Perceptron Problem

Overview

Gradient descent is used to optimize the perceptron's weights by minimizing the error between predicted and actual outputs. In our implementation, we use gradient descent to iteratively adjust the weights and bias to find the optimal decision boundary.

How it Works

Let's start with one perceptron with 2 input nodes. The input nodes have feature values x_1 and x_2. The perceptron has three weights: w_1,w_2, and b. We will be using a sigmoid activation function.

1. **Forward Pass**:
    - Input values are multiplied by weights: $z = w_1x_1 + w_2x_2 + b$
    - Result is passed through sigmoid activation: $\sigma(z) = 1/(1 + e^{(-z)})$
    - This gives us our prediction $\hat{y}$
2. **Error Calculation**:
    - We use Mean Squared Error (MSE): $E = ½(y - \hat{y})^2$
    - y is the true label (0 or 1)
    - $\hat{y}$ is our prediction (between 0 and 1)
3. **Backward Pass (Gradient Descent)**:
    - We calculate how much each weight contributed to the error
    - For weights: $\partial E/\partial w = (y - \hat{y}) \times x \times \hat{y}(1-\hat{y})$

- For bias: ∂E/∂b = (y - ŷ) × ŷ(1-ŷ)
- The term ŷ(1-ŷ) comes from derivative of sigmoid function

4. **Weight Update**:

- Weights are updated using learning rate (α):
- w_new = w_old + α × ∂E/∂w
- b_new = b_old + α × ∂E/∂b
- Learning rate determines step size of updates

Visualization

In our implementation, we can see gradient descent in action through:

1. Decision boundary plot before and after training
2. Loss curve showing error reduction over time
3. Final classification regions showing learned decision boundary

Key Parameters

- Learning rate (default = 0.01): Controls how big steps we take
- Epochs (default = 100): Number of passes through training data
- These hyperparameters affect:
  - Speed of convergence
  - Stability of training
  - Quality of final solution

Understanding gradient descent is crucial as it forms the foundation for training more complex neural networks.

Building a perceptron

Below is some code that creates a Preceptron class with a single perceptron using the sigmoid activation function. Since the derivative of the sigmoid function is analytic, computing the gradient for gradient descent is straightfoward.

Below is some example code for building a perceptron

```
import numpy as np
import matplotlib.pyplot as plt

class Perceptron:
    def __init__(self, input_size):
        """
        Initialize a perceptron with given input dimensions
```

```
        Args:
            input_size: Number of input features
        """
        # Initialize weights using normal distribution for
better initial convergence
        self.weights = np.random.randn(input_size)
        # Initialize bias term (equivalent to threshold in
basic perceptron)
        self.bias = np.random.randn()

    def forward(self, x):
        """
        Forward pass computes weighted sum and applies
activation
        Args:
            x: Input features array
        Returns:
            Output after sigmoid activation
        """
        # Compute weighted sum: w^T * x + b

        # IMPLEMENT HERE
        z = ...

        # Apply sigmoid activation: f(z) = 1/(1 + e^(-z))
        # Sigmoid maps any input to (0,1) range, useful for
binary classification
        f = ...

        # return the value after activation
        return f

    def backward(self, x, y, output):
        """
        Compute gradients for weight update using chain rule
        Args:
            x: Input features
            y: True label
            output: Predicted output from forward pass
        Returns:
            Gradients for weights and bias
        """
        # Error term (y - ŷ)
        error = y - output
        # Gradient includes:
        # 1. Error term
        # 2. Derivative of sigmoid: f'(x) = f(x)(1-f(x))
```

Question 2 (4 pts)

1. Use the example code or implement your own perceptron with sigmoid activation function. If using the example code, I recommend manually typing it out to get a feel for the code and syntax. (2 pts)
2. Show an example of using the forward pass to predict a value of Y given X with the intial randomized weights. (1 pt)
3. What are range of possible outputs values for this perceptron? (1 pt)

In [ ]:
```
# use the example code or implement your own perceptron with sigmoid
activation function. If using the example code,fill in the forward()
function to compute the forward pass.
```

2.1 Implementation of perceptron !

In [7]:
```
import numpy as np
import matplotlib.pyplot as plt

class Perceptron:
    def __init__(self, input_size):
        """
        Initialize a perceptron with given input dimensions
        Args:
            input_size: Number of input features
        """
        # Initialize weights using normal distribution for better initial
convergence
        self.weights = np.random.randn(input_size)
        # Initialize bias term (equivalent to threshold in basic
perceptron)
        self.bias = np.random.randn()


    def forward(self, x):
        """
        Forward pass computes weighted sum and applies activation
        Args:
            x: Input features array
        Returns:
            Output after sigmoid activation
        """
        # Compute weighted sum: w^T * x + b

        # IMPLEMENT HERE
        z = np.dot(self.weights,x)+b

        # Apply sigmoid activation: f(z) = 1/(1 + e^(-z))
        # Sigmoid maps any input to (0,1) range, useful for binary
classification
        f = 1/(1+np.exp(-z))

        # return the value after activation
        return f
```

```
    def backward(self, x, y, output):
        """
        Compute gradients for weight update using chain rule
        Args:
            x: Input features
            y: True label
            output: Predicted output from forward pass
        Returns:
            Gradients for weights and bias
        """
        # Error term (y - ŷ)
        error = y - output
        # Gradient includes:
        D_w = np.dot(error,output*(1-output))*x
        D_b = np.dot(error,output*(1-output))
        # 1. Error term
        # 2. Derivative of sigmoid: f'(x) = f(x)(1-f(x))

        return D_w, D_b
```

Training a perceptron

To train a perceptron, we will use backpropagation. For training, one needs to decide on the hyperparameters such as the learning rate and the number of epochs to train for.

Below is some example code for implementing the training

First, let's create an artificial dataset for training and evaluation

In [8]:
```
# generate features for binary classification
np.random.seed(12) # set seed for reproducibility
X = np.random.randn(200, 2)  # 200 samples total
y = (X[:, 0] + X[:, 1] > 0).astype(float)

# Split into training and validation sets (80-20 split)
split_idx = int(0.8 * len(X))
X_train, X_val = X[:split_idx], X[split_idx:]
y_train, y_val = y[:split_idx], y[split_idx:]
```

Then we'll need to set some hyperparameters

In [9]:
```
# Hyperparameters
learning_rate = 0.1
epochs = 100
```

Now, we can create and train the perceptron. You can use the code below or create your own code to train and plot the results

```
# Create and train the perceptron
perceptron = Perceptron(input_size=2)
```

```
def calculate_loss(model, X, y):
    total_loss = 0
    for i in range(len(X)):
        output = model.forward(X[i])
        total_loss += 0.5 * (y[i] - output)**2
    return total_loss / len(X)

# Training with validation
training_losses = []
validation_losses = []

for epoch in range(epochs):
    epoch_loss = 0

    # Training pass
    for i in range(len(X_train)):
        output = perceptron.forward(X_train[i])
        loss = 0.5 * (y_train[i] - output)**2
        epoch_loss += loss

        d_weights, d_bias = perceptron.backward(X_train[i],
y_train[i], output)
        perceptron.weights += learning_rate* d_weights
        perceptron.bias += learning_rate * d_bias

    # Calculate average losses
    train_loss = epoch_loss / len(X_train)
    val_loss = calculate_loss(perceptron, X_val, y_val)

    training_losses.append(train_loss)
    validation_losses.append(val_loss)

    if epoch % 10 == 0:
        print(f'Epoch {epoch}, Train Loss: {train_loss:.4f},
Val Loss: {val_loss:.4f}')

# Plot training and validation losses
plt.figure(figsize=(10, 5))
plt.plot(training_losses, label='Training Loss')
plt.plot(validation_losses, label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training and Validation Loss over Time')
plt.legend()
plt.show()

# Plot decision boundaries with both training and validation
data
plt.figure(figsize=(10, 8))
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
```

```
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100),
                         np.linspace(y_min, y_max, 100))

    Z = np.zeros(xx.shape)
    for i in range(xx.shape[0]):
        for j in range(xx.shape[1]):
            Z[i, j] = perceptron.forward(np.array([xx[i, j],
    yy[i, j]]))

    plt.contourf(xx, yy, Z, levels=[0, 0.5, 1],
    cmap=plt.cm.RdYlBu, alpha=0.6)

    # Plot training data
    plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train,
                cmap=plt.cm.RdYlBu, marker='o', label='Training
    Data')

    # Plot validation data
    plt.scatter(X_val[:, 0], X_val[:, 1], c=y_val,
                cmap=plt.cm.RdYlBu, marker='^', label='Validation
    Data')

    plt.xlabel('Feature 1')
    plt.ylabel('Feature 2')
    plt.title('Decision Boundary with Training and Validation
    Data')
    plt.legend()
    plt.colorbar()
    plt.show()
```

In [10]:

```
# Create and train the perceptron
perceptron = Perceptron(input_size=2)

def calculate_loss(model, X, y):
    total_loss = 0
    for i in range(len(X)):
        output = model.forward(X[i])
        total_loss += 0.5 * (y[i] - output)**2
    return total_loss / len(X)

# Training with validation
training_losses = []
validation_losses = []

for epoch in range(epochs):
    epoch_loss = 0

    # Training pass
    for i in range(len(X_train)):
        output = perceptron.forward(X_train[i])
        loss = 0.5 * (y_train[i] - output)**2
        epoch_loss += loss
```

```
        d_weights, d_bias = perceptron.backward(X_train[i], y_train[i],
output)
        perceptron.weights += learning_rate* d_weights
        perceptron.bias += learning_rate * d_bias

    # Calculate average losses
    train_loss = epoch_loss / len(X_train)
    val_loss = calculate_loss(perceptron, X_val, y_val)

    training_losses.append(train_loss)
    validation_losses.append(val_loss)

    if epoch % 10 == 0:
        print(f'Epoch {epoch}, Train Loss: {train_loss:.4f}, Val Loss:
{val_loss:.4f}')

# Plot training and validation losses
plt.figure(figsize=(10, 5))
plt.plot(training_losses, label='Training Loss')
plt.plot(validation_losses, label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training and Validation Loss over Time')
plt.legend()
plt.show()

# Plot decision boundaries with both training and validation data
plt.figure(figsize=(10, 8))
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100),
                     np.linspace(y_min, y_max, 100))

Z = np.zeros(xx.shape)
for i in range(xx.shape[0]):
    for j in range(xx.shape[1]):
        Z[i, j] = perceptron.forward(np.array([xx[i, j], yy[i, j]]))

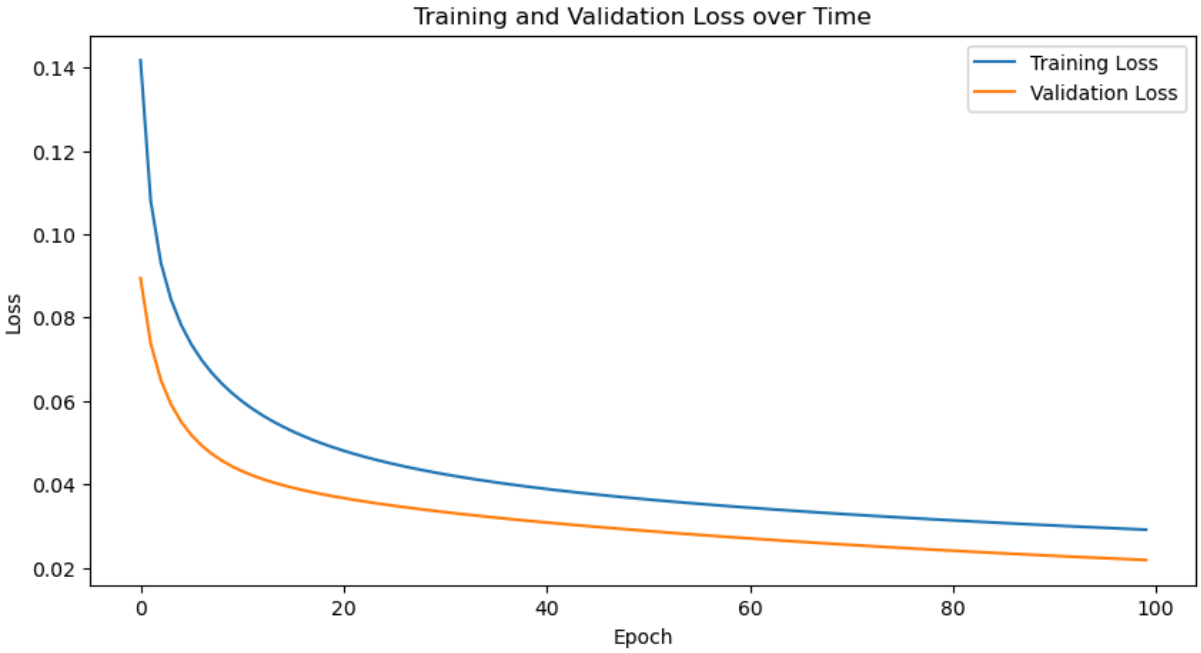
plt.contourf(xx, yy, Z, levels=[0, 0.5, 1], cmap=plt.cm.RdYlBu, alpha=0.6)

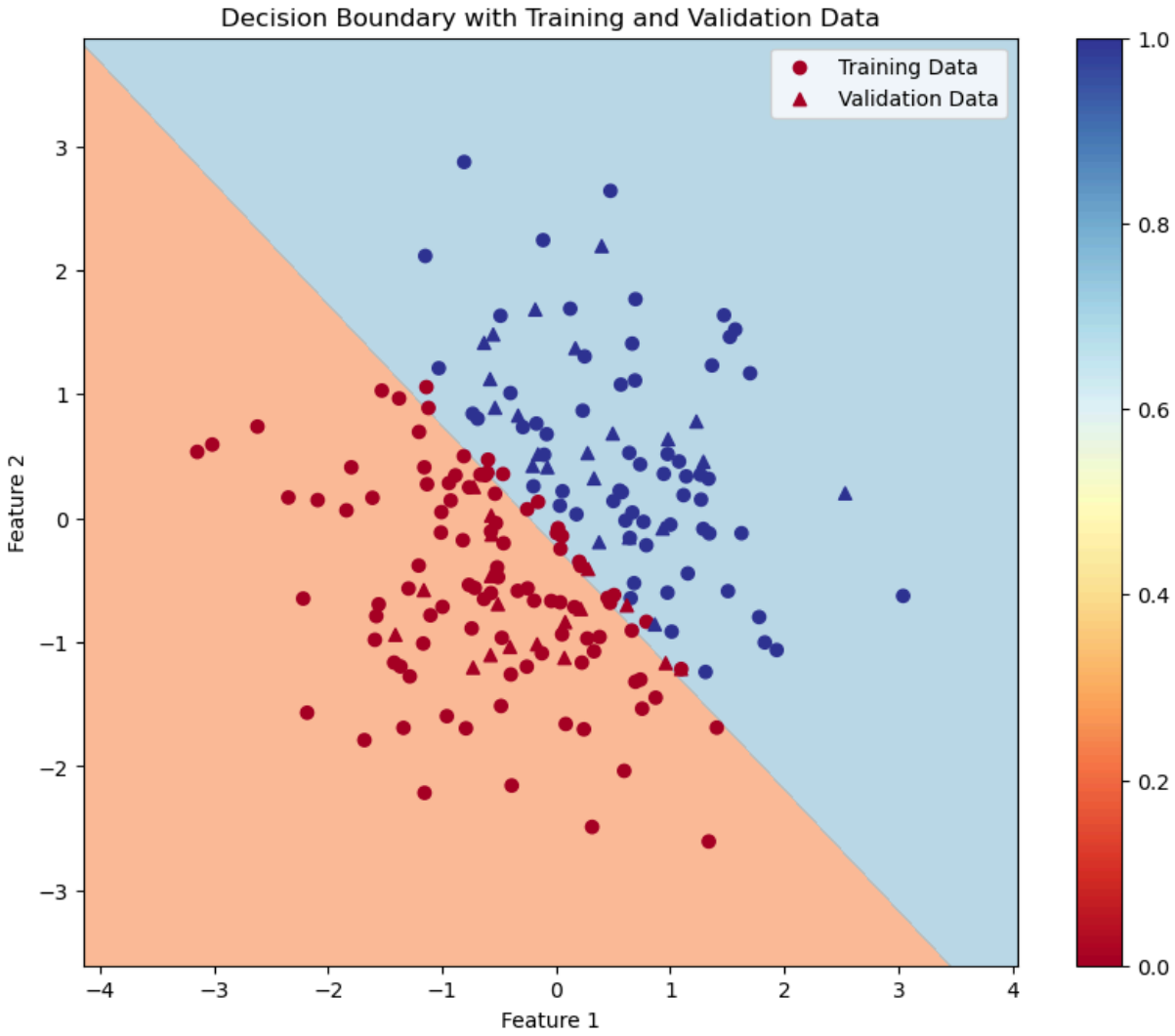
# Plot training data
plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train,
           cmap=plt.cm.RdYlBu, marker='o', label='Training Data')

# Plot validation data
plt.scatter(X_val[:, 0], X_val[:, 1], c=y_val,
           cmap=plt.cm.RdYlBu, marker='^', label='Validation Data')

plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Decision Boundary with Training and Validation Data')
plt.legend()
plt.colorbar()
plt.show()
```

```
Epoch 0, Train Loss: 0.1417, Val Loss: 0.0894
Epoch 10, Train Loss: 0.0599, Val Loss: 0.0432
Epoch 20, Train Loss: 0.0481, Val Loss: 0.0367
Epoch 30, Train Loss: 0.0424, Val Loss: 0.0333
Epoch 40, Train Loss: 0.0389, Val Loss: 0.0308
Epoch 50, Train Loss: 0.0364, Val Loss: 0.0288
Epoch 60, Train Loss: 0.0344, Val Loss: 0.0270
Epoch 70, Train Loss: 0.0327, Val Loss: 0.0254
Epoch 80, Train Loss: 0.0313, Val Loss: 0.0241
Epoch 90, Train Loss: 0.0301, Val Loss: 0.0228
```





Change the number of epochs to 1000

In [12]:
```
learning_rate = 0.1
epochs = 1000
# Create and train the perceptron
perceptron = Perceptron(input_size=2)

def calculate_loss(model, X, y):
    total_loss = 0
    for i in range(len(X)):
        output = model.forward(X[i])
        total_loss += 0.5 * (y[i] - output)**2
    return total_loss / len(X)

# Training with validation
training_losses = []
validation_losses = []

for epoch in range(epochs):
    epoch_loss = 0

    # Training pass
```

```
    for i in range(len(X_train)):
        output = perceptron.forward(X_train[i])
        loss = 0.5 * (y_train[i] - output)**2
        epoch_loss += loss

        d_weights, d_bias = perceptron.backward(X_train[i], y_train[i],
output)
        perceptron.weights += learning_rate* d_weights
        perceptron.bias += learning_rate * d_bias

    # Calculate average losses
    train_loss = epoch_loss / len(X_train)
    val_loss = calculate_loss(perceptron, X_val, y_val)

    training_losses.append(train_loss)
    validation_losses.append(val_loss)

    if epoch % 10 == 0:
        print(f'Epoch {epoch}, Train Loss: {train_loss:.4f}, Val Loss:
{val_loss:.4f}')

# Plot training and validation losses
plt.figure(figsize=(10, 5))
plt.plot(training_losses, label='Training Loss')
plt.plot(validation_losses, label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training and Validation Loss over Time')
plt.legend()
plt.show()

# Plot decision boundaries with both training and validation data
plt.figure(figsize=(10, 8))
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100),
                     np.linspace(y_min, y_max, 100))

Z = np.zeros(xx.shape)
for i in range(xx.shape[0]):
    for j in range(xx.shape[1]):
        Z[i, j] = perceptron.forward(np.array([xx[i, j], yy[i, j]]))

plt.contourf(xx, yy, Z, levels=[0, 0.5, 1], cmap=plt.cm.RdYlBu, alpha=0.6)

# Plot training data
plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train,
           cmap=plt.cm.RdYlBu, marker='o', label='Training Data')

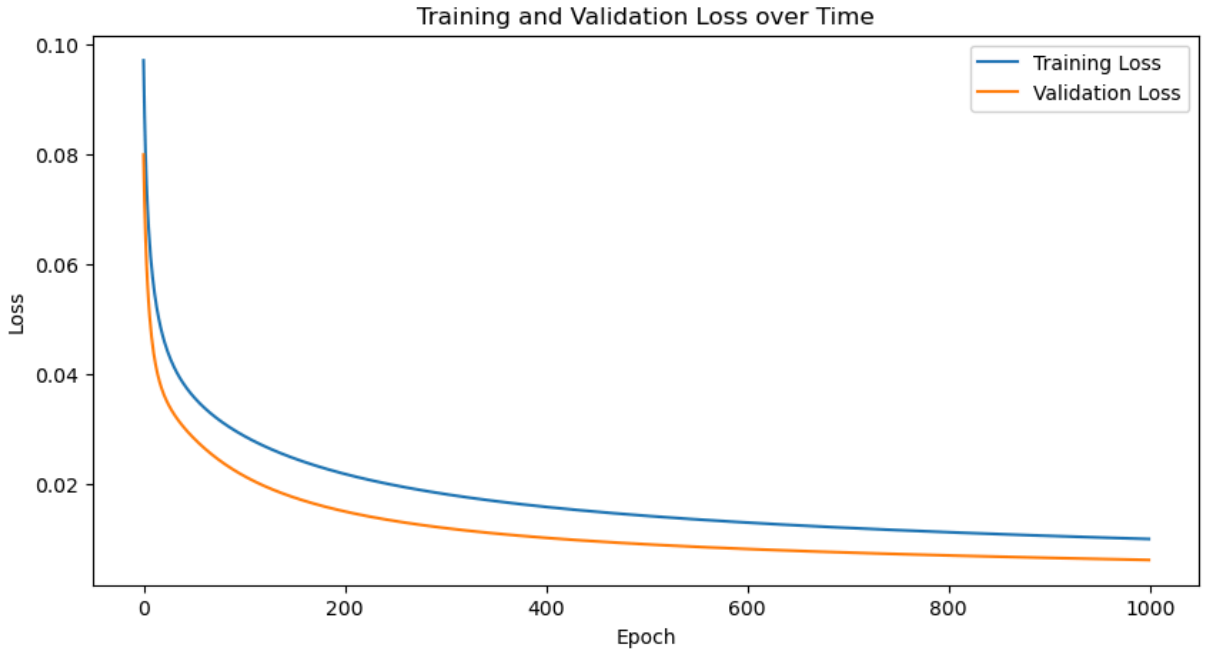
# Plot validation data
plt.scatter(X_val[:, 0], X_val[:, 1], c=y_val,
           cmap=plt.cm.RdYlBu, marker='^', label='Validation Data')

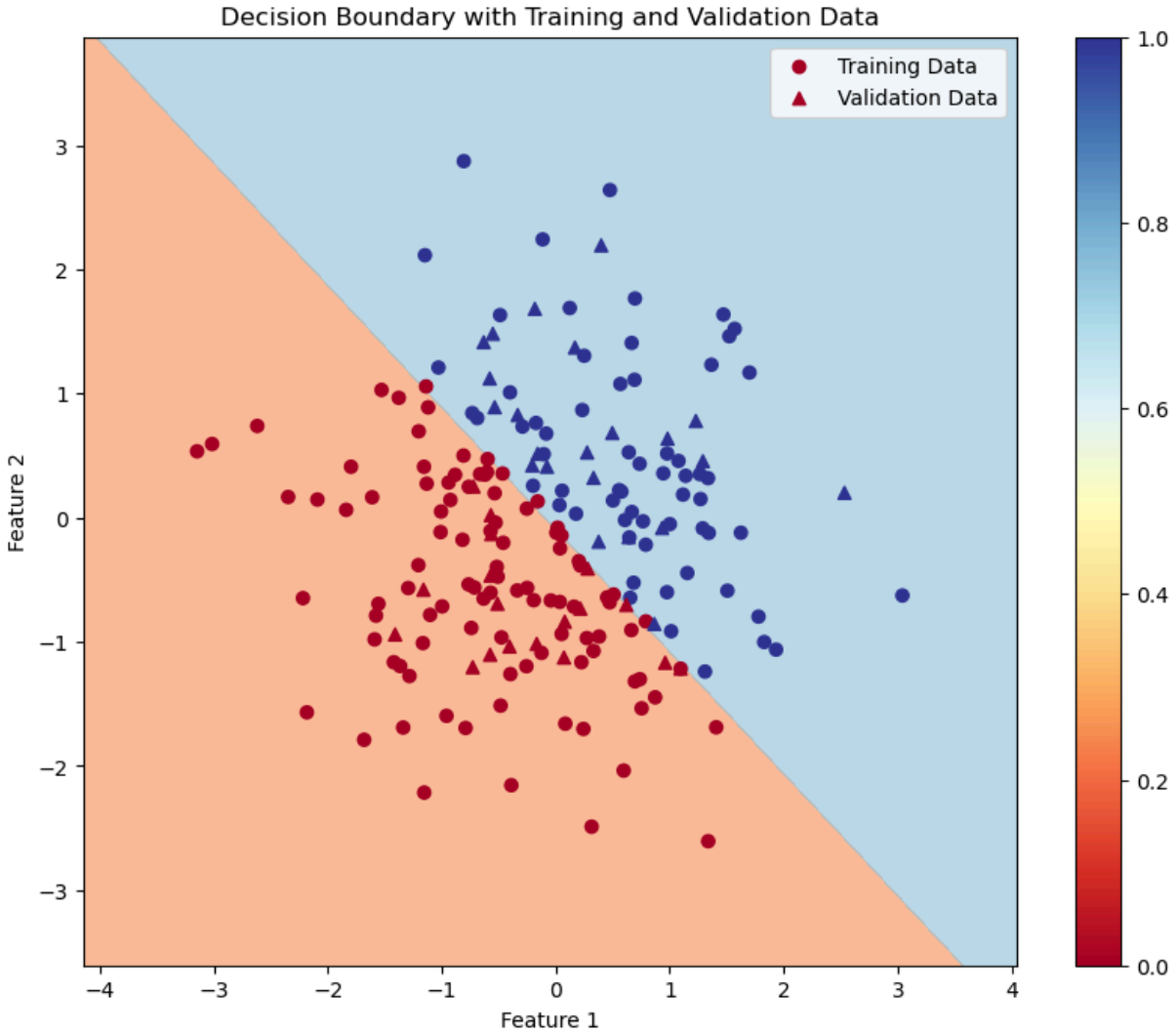
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Decision Boundary with Training and Validation Data')
```

```
plt.legend()
plt.colorbar()
plt.show()
```

```
Epoch 0, Train Loss: 0.0970, Val Loss: 0.0799
Epoch 10, Train Loss: 0.0560, Val Loss: 0.0439
Epoch 20, Train Loss: 0.0463, Val Loss: 0.0364
Epoch 30, Train Loss: 0.0414, Val Loss: 0.0328
Epoch 40, Train Loss: 0.0382, Val Loss: 0.0303
Epoch 50, Train Loss: 0.0358, Val Loss: 0.0283
Epoch 60, Train Loss: 0.0340, Val Loss: 0.0266
Epoch 70, Train Loss: 0.0324, Val Loss: 0.0251
Epoch 80, Train Loss: 0.0310, Val Loss: 0.0238
Epoch 90, Train Loss: 0.0298, Val Loss: 0.0226
Epoch 100, Train Loss: 0.0288, Val Loss: 0.0215
Epoch 110, Train Loss: 0.0278, Val Loss: 0.0205
Epoch 120, Train Loss: 0.0269, Val Loss: 0.0197
Epoch 130, Train Loss: 0.0261, Val Loss: 0.0189
Epoch 140, Train Loss: 0.0254, Val Loss: 0.0182
Epoch 150, Train Loss: 0.0247, Val Loss: 0.0176
Epoch 160, Train Loss: 0.0240, Val Loss: 0.0170
Epoch 170, Train Loss: 0.0234, Val Loss: 0.0164
Epoch 180, Train Loss: 0.0229, Val Loss: 0.0159
Epoch 190, Train Loss: 0.0224, Val Loss: 0.0155
Epoch 200, Train Loss: 0.0219, Val Loss: 0.0151
Epoch 210, Train Loss: 0.0214, Val Loss: 0.0147
Epoch 220, Train Loss: 0.0210, Val Loss: 0.0143
Epoch 230, Train Loss: 0.0206, Val Loss: 0.0139
Epoch 240, Train Loss: 0.0202, Val Loss: 0.0136
Epoch 250, Train Loss: 0.0198, Val Loss: 0.0133
Epoch 260, Train Loss: 0.0194, Val Loss: 0.0130
Epoch 270, Train Loss: 0.0191, Val Loss: 0.0128
Epoch 280, Train Loss: 0.0188, Val Loss: 0.0125
Epoch 290, Train Loss: 0.0185, Val Loss: 0.0123
Epoch 300, Train Loss: 0.0182, Val Loss: 0.0120
Epoch 310, Train Loss: 0.0179, Val Loss: 0.0118
Epoch 320, Train Loss: 0.0177, Val Loss: 0.0116
Epoch 330, Train Loss: 0.0174, Val Loss: 0.0114
Epoch 340, Train Loss: 0.0172, Val Loss: 0.0112
Epoch 350, Train Loss: 0.0169, Val Loss: 0.0110
Epoch 360, Train Loss: 0.0167, Val Loss: 0.0109
Epoch 370, Train Loss: 0.0165, Val Loss: 0.0107
Epoch 380, Train Loss: 0.0163, Val Loss: 0.0106
Epoch 390, Train Loss: 0.0161, Val Loss: 0.0104
Epoch 400, Train Loss: 0.0159, Val Loss: 0.0103
Epoch 410, Train Loss: 0.0157, Val Loss: 0.0101
Epoch 420, Train Loss: 0.0155, Val Loss: 0.0100
Epoch 430, Train Loss: 0.0153, Val Loss: 0.0099
Epoch 440, Train Loss: 0.0152, Val Loss: 0.0097
Epoch 450, Train Loss: 0.0150, Val Loss: 0.0096
Epoch 460, Train Loss: 0.0148, Val Loss: 0.0095
Epoch 470, Train Loss: 0.0147, Val Loss: 0.0094
Epoch 480, Train Loss: 0.0145, Val Loss: 0.0093
Epoch 490, Train Loss: 0.0144, Val Loss: 0.0092
Epoch 500, Train Loss: 0.0143, Val Loss: 0.0091
Epoch 510, Train Loss: 0.0141, Val Loss: 0.0090
Epoch 520, Train Loss: 0.0140, Val Loss: 0.0089
Epoch 530, Train Loss: 0.0139, Val Loss: 0.0088
Epoch 540, Train Loss: 0.0137, Val Loss: 0.0087
Epoch 550, Train Loss: 0.0136, Val Loss: 0.0086
```

```
Epoch 560, Train Loss: 0.0135, Val Loss: 0.0085
Epoch 570, Train Loss: 0.0134, Val Loss: 0.0085
Epoch 580, Train Loss: 0.0133, Val Loss: 0.0084
Epoch 590, Train Loss: 0.0131, Val Loss: 0.0083
Epoch 600, Train Loss: 0.0130, Val Loss: 0.0082
Epoch 610, Train Loss: 0.0129, Val Loss: 0.0082
Epoch 620, Train Loss: 0.0128, Val Loss: 0.0081
Epoch 630, Train Loss: 0.0127, Val Loss: 0.0080
Epoch 640, Train Loss: 0.0126, Val Loss: 0.0080
Epoch 650, Train Loss: 0.0125, Val Loss: 0.0079
Epoch 660, Train Loss: 0.0124, Val Loss: 0.0078
Epoch 670, Train Loss: 0.0123, Val Loss: 0.0078
Epoch 680, Train Loss: 0.0122, Val Loss: 0.0077
Epoch 690, Train Loss: 0.0122, Val Loss: 0.0076
Epoch 700, Train Loss: 0.0121, Val Loss: 0.0076
Epoch 710, Train Loss: 0.0120, Val Loss: 0.0075
Epoch 720, Train Loss: 0.0119, Val Loss: 0.0075
Epoch 730, Train Loss: 0.0118, Val Loss: 0.0074
Epoch 740, Train Loss: 0.0117, Val Loss: 0.0074
Epoch 750, Train Loss: 0.0117, Val Loss: 0.0073
Epoch 760, Train Loss: 0.0116, Val Loss: 0.0073
Epoch 770, Train Loss: 0.0115, Val Loss: 0.0072
Epoch 780, Train Loss: 0.0114, Val Loss: 0.0072
Epoch 790, Train Loss: 0.0114, Val Loss: 0.0071
Epoch 800, Train Loss: 0.0113, Val Loss: 0.0071
Epoch 810, Train Loss: 0.0112, Val Loss: 0.0070
Epoch 820, Train Loss: 0.0111, Val Loss: 0.0070
Epoch 830, Train Loss: 0.0111, Val Loss: 0.0069
Epoch 840, Train Loss: 0.0110, Val Loss: 0.0069
Epoch 850, Train Loss: 0.0109, Val Loss: 0.0068
Epoch 860, Train Loss: 0.0109, Val Loss: 0.0068
Epoch 870, Train Loss: 0.0108, Val Loss: 0.0067
Epoch 880, Train Loss: 0.0107, Val Loss: 0.0067
Epoch 890, Train Loss: 0.0107, Val Loss: 0.0067
Epoch 900, Train Loss: 0.0106, Val Loss: 0.0066
Epoch 910, Train Loss: 0.0106, Val Loss: 0.0066
Epoch 920, Train Loss: 0.0105, Val Loss: 0.0065
Epoch 930, Train Loss: 0.0104, Val Loss: 0.0065
Epoch 940, Train Loss: 0.0104, Val Loss: 0.0065
Epoch 950, Train Loss: 0.0103, Val Loss: 0.0064
Epoch 960, Train Loss: 0.0103, Val Loss: 0.0064
Epoch 970, Train Loss: 0.0102, Val Loss: 0.0063
Epoch 980, Train Loss: 0.0102, Val Loss: 0.0063
Epoch 990, Train Loss: 0.0101, Val Loss: 0.0063
```





The training and validation losses go down systematically with the epochs. In fact the validation loss is lower than the training loss which means that the model is generalizing to the validation data set extremely well !

Choose a very small learning rate of 0.001 - the losses are higher !

In [15]:
```
learning_rate = 0.001
epochs = 1000
# Create and train the perceptron
perceptron = Perceptron(input_size=2)

def calculate_loss(model, X, y):
    total_loss = 0
    for i in range(len(X)):
        output = model.forward(X[i])
        total_loss += 0.5 * (y[i] - output)**2
    return total_loss / len(X)

# Training with validation
training_losses = []
validation_losses = []

for epoch in range(epochs):
    epoch_loss = 0

    # Training pass
    for i in range(len(X_train)):
        output = perceptron.forward(X_train[i])
        loss = 0.5 * (y_train[i] - output)**2
        epoch_loss += loss

        d_weights, d_bias = perceptron.backward(X_train[i], y_train[i],
output)
        perceptron.weights += learning_rate* d_weights
        perceptron.bias += learning_rate * d_bias

    # Calculate average losses
    train_loss = epoch_loss / len(X_train)
    val_loss = calculate_loss(perceptron, X_val, y_val)

    training_losses.append(train_loss)
    validation_losses.append(val_loss)

    if epoch % 10 == 0:
        print(f'Epoch {epoch}, Train Loss: {train_loss:.4f}, Val Loss:
{val_loss:.4f}')

# Plot training and validation losses
plt.figure(figsize=(10, 5))
plt.plot(training_losses, label='Training Loss')
plt.plot(validation_losses, label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training and Validation Loss over Time')
plt.legend()
plt.show()
```

```
# Plot decision boundaries with both training and validation data
plt.figure(figsize=(10, 8))
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100),
                     np.linspace(y_min, y_max, 100))

Z = np.zeros(xx.shape)
for i in range(xx.shape[0]):
    for j in range(xx.shape[1]):
        Z[i, j] = perceptron.forward(np.array([xx[i, j], yy[i, j]]))

plt.contourf(xx, yy, Z, levels=[0, 0.5, 1], cmap=plt.cm.RdYlBu, alpha=0.6)

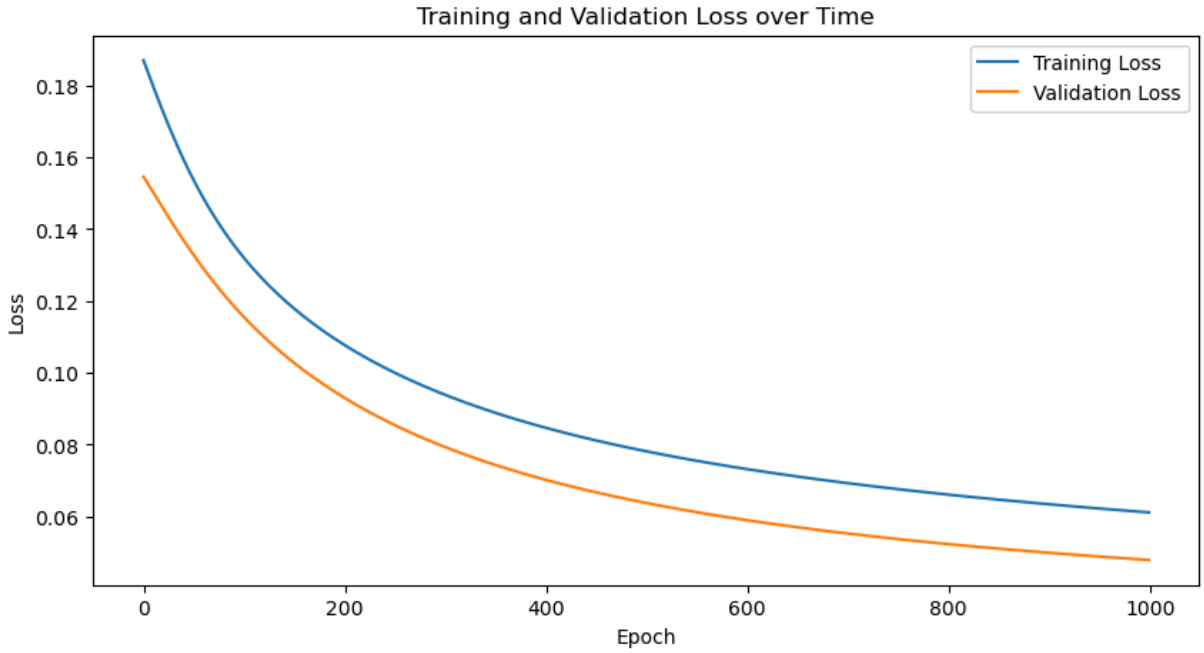
# Plot training data
plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train,
           cmap=plt.cm.RdYlBu, marker='o', label='Training Data')

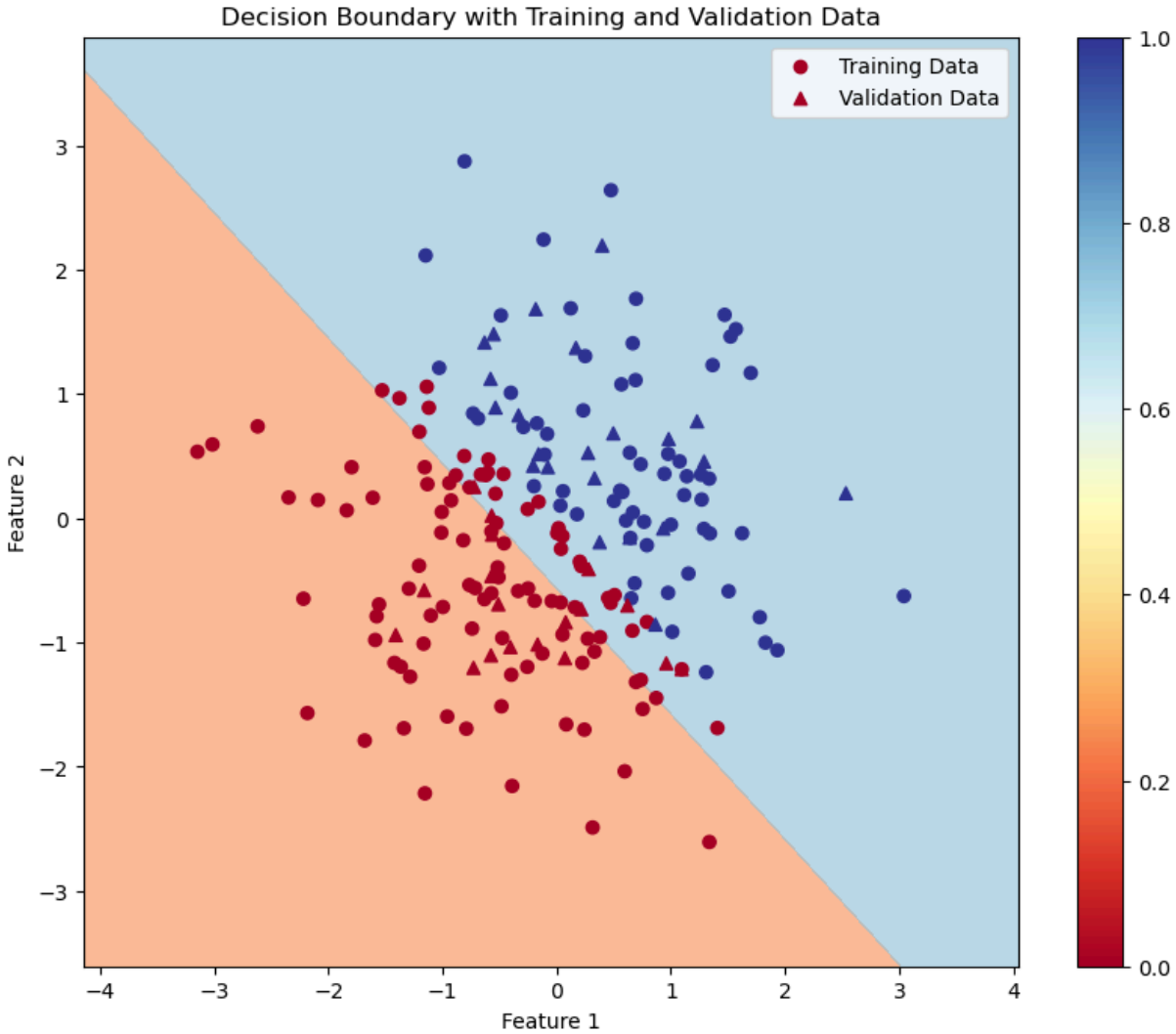
# Plot validation data
plt.scatter(X_val[:, 0], X_val[:, 1], c=y_val,
           cmap=plt.cm.RdYlBu, marker='^', label='Validation Data')

plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Decision Boundary with Training and Validation Data')
plt.legend()
plt.colorbar()
plt.show()
```

```
Epoch 0, Train Loss: 0.1869, Val Loss: 0.1546
Epoch 10, Train Loss: 0.1794, Val Loss: 0.1500
Epoch 20, Train Loss: 0.1722, Val Loss: 0.1455
Epoch 30, Train Loss: 0.1654, Val Loss: 0.1411
Epoch 40, Train Loss: 0.1592, Val Loss: 0.1368
Epoch 50, Train Loss: 0.1535, Val Loss: 0.1327
Epoch 60, Train Loss: 0.1483, Val Loss: 0.1288
Epoch 70, Train Loss: 0.1436, Val Loss: 0.1252
Epoch 80, Train Loss: 0.1394, Val Loss: 0.1217
Epoch 90, Train Loss: 0.1355, Val Loss: 0.1185
Epoch 100, Train Loss: 0.1319, Val Loss: 0.1154
Epoch 110, Train Loss: 0.1286, Val Loss: 0.1126
Epoch 120, Train Loss: 0.1256, Val Loss: 0.1099
Epoch 130, Train Loss: 0.1228, Val Loss: 0.1073
Epoch 140, Train Loss: 0.1202, Val Loss: 0.1049
Epoch 150, Train Loss: 0.1178, Val Loss: 0.1027
Epoch 160, Train Loss: 0.1155, Val Loss: 0.1005
Epoch 170, Train Loss: 0.1134, Val Loss: 0.0985
Epoch 180, Train Loss: 0.1114, Val Loss: 0.0966
Epoch 190, Train Loss: 0.1095, Val Loss: 0.0947
Epoch 200, Train Loss: 0.1077, Val Loss: 0.0930
Epoch 210, Train Loss: 0.1060, Val Loss: 0.0913
Epoch 220, Train Loss: 0.1044, Val Loss: 0.0897
Epoch 230, Train Loss: 0.1028, Val Loss: 0.0882
Epoch 240, Train Loss: 0.1014, Val Loss: 0.0868
Epoch 250, Train Loss: 0.1000, Val Loss: 0.0854
Epoch 260, Train Loss: 0.0986, Val Loss: 0.0841
Epoch 270, Train Loss: 0.0973, Val Loss: 0.0828
Epoch 280, Train Loss: 0.0961, Val Loss: 0.0816
Epoch 290, Train Loss: 0.0949, Val Loss: 0.0804
Epoch 300, Train Loss: 0.0938, Val Loss: 0.0793
Epoch 310, Train Loss: 0.0927, Val Loss: 0.0782
Epoch 320, Train Loss: 0.0917, Val Loss: 0.0772
Epoch 330, Train Loss: 0.0907, Val Loss: 0.0762
Epoch 340, Train Loss: 0.0897, Val Loss: 0.0752
Epoch 350, Train Loss: 0.0888, Val Loss: 0.0743
Epoch 360, Train Loss: 0.0879, Val Loss: 0.0734
Epoch 370, Train Loss: 0.0871, Val Loss: 0.0726
Epoch 380, Train Loss: 0.0862, Val Loss: 0.0717
Epoch 390, Train Loss: 0.0854, Val Loss: 0.0709
Epoch 400, Train Loss: 0.0847, Val Loss: 0.0702
Epoch 410, Train Loss: 0.0839, Val Loss: 0.0694
Epoch 420, Train Loss: 0.0832, Val Loss: 0.0687
Epoch 430, Train Loss: 0.0825, Val Loss: 0.0680
Epoch 440, Train Loss: 0.0818, Val Loss: 0.0674
Epoch 450, Train Loss: 0.0811, Val Loss: 0.0667
Epoch 460, Train Loss: 0.0805, Val Loss: 0.0661
Epoch 470, Train Loss: 0.0799, Val Loss: 0.0655
Epoch 480, Train Loss: 0.0793, Val Loss: 0.0649
Epoch 490, Train Loss: 0.0787, Val Loss: 0.0643
Epoch 500, Train Loss: 0.0781, Val Loss: 0.0637
Epoch 510, Train Loss: 0.0776, Val Loss: 0.0632
Epoch 520, Train Loss: 0.0770, Val Loss: 0.0627
Epoch 530, Train Loss: 0.0765, Val Loss: 0.0622
Epoch 540, Train Loss: 0.0760, Val Loss: 0.0617
Epoch 550, Train Loss: 0.0755, Val Loss: 0.0612
```

```
Epoch 560, Train Loss: 0.0750, Val Loss: 0.0607
Epoch 570, Train Loss: 0.0745, Val Loss: 0.0603
Epoch 580, Train Loss: 0.0741, Val Loss: 0.0598
Epoch 590, Train Loss: 0.0736, Val Loss: 0.0594
Epoch 600, Train Loss: 0.0732, Val Loss: 0.0590
Epoch 610, Train Loss: 0.0727, Val Loss: 0.0585
Epoch 620, Train Loss: 0.0723, Val Loss: 0.0581
Epoch 630, Train Loss: 0.0719, Val Loss: 0.0577
Epoch 640, Train Loss: 0.0715, Val Loss: 0.0574
Epoch 650, Train Loss: 0.0711, Val Loss: 0.0570
Epoch 660, Train Loss: 0.0707, Val Loss: 0.0566
Epoch 670, Train Loss: 0.0703, Val Loss: 0.0563
Epoch 680, Train Loss: 0.0700, Val Loss: 0.0559
Epoch 690, Train Loss: 0.0696, Val Loss: 0.0556
Epoch 700, Train Loss: 0.0693, Val Loss: 0.0553
Epoch 710, Train Loss: 0.0689, Val Loss: 0.0549
Epoch 720, Train Loss: 0.0686, Val Loss: 0.0546
Epoch 730, Train Loss: 0.0682, Val Loss: 0.0543
Epoch 740, Train Loss: 0.0679, Val Loss: 0.0540
Epoch 750, Train Loss: 0.0676, Val Loss: 0.0537
Epoch 760, Train Loss: 0.0673, Val Loss: 0.0534
Epoch 770, Train Loss: 0.0670, Val Loss: 0.0531
Epoch 780, Train Loss: 0.0667, Val Loss: 0.0528
Epoch 790, Train Loss: 0.0664, Val Loss: 0.0526
Epoch 800, Train Loss: 0.0661, Val Loss: 0.0523
Epoch 810, Train Loss: 0.0658, Val Loss: 0.0520
Epoch 820, Train Loss: 0.0655, Val Loss: 0.0518
Epoch 830, Train Loss: 0.0652, Val Loss: 0.0515
Epoch 840, Train Loss: 0.0649, Val Loss: 0.0513
Epoch 850, Train Loss: 0.0647, Val Loss: 0.0510
Epoch 860, Train Loss: 0.0644, Val Loss: 0.0508
Epoch 870, Train Loss: 0.0642, Val Loss: 0.0506
Epoch 880, Train Loss: 0.0639, Val Loss: 0.0503
Epoch 890, Train Loss: 0.0636, Val Loss: 0.0501
Epoch 900, Train Loss: 0.0634, Val Loss: 0.0499
Epoch 910, Train Loss: 0.0631, Val Loss: 0.0497
Epoch 920, Train Loss: 0.0629, Val Loss: 0.0495
Epoch 930, Train Loss: 0.0627, Val Loss: 0.0492
Epoch 940, Train Loss: 0.0624, Val Loss: 0.0490
Epoch 950, Train Loss: 0.0622, Val Loss: 0.0488
Epoch 960, Train Loss: 0.0620, Val Loss: 0.0486
Epoch 970, Train Loss: 0.0618, Val Loss: 0.0484
Epoch 980, Train Loss: 0.0615, Val Loss: 0.0483
Epoch 990, Train Loss: 0.0613, Val Loss: 0.0481
```





Choose a large learning rate of 10 - the losses are smaller !

In [16]:
```
learning_rate = 10
epochs = 1000
# Create and train the perceptron
perceptron = Perceptron(input_size=2)

def calculate_loss(model, X, y):
    total_loss = 0
    for i in range(len(X)):
        output = model.forward(X[i])
        total_loss += 0.5 * (y[i] - output)**2
    return total_loss / len(X)

# Training with validation
training_losses = []
validation_losses = []

for epoch in range(epochs):
    epoch_loss = 0

    # Training pass
    for i in range(len(X_train)):
        output = perceptron.forward(X_train[i])
        loss = 0.5 * (y_train[i] - output)**2
        epoch_loss += loss

        d_weights, d_bias = perceptron.backward(X_train[i], y_train[i],
output)
        perceptron.weights += learning_rate* d_weights
        perceptron.bias += learning_rate * d_bias

    # Calculate average losses
    train_loss = epoch_loss / len(X_train)
    val_loss = calculate_loss(perceptron, X_val, y_val)

    training_losses.append(train_loss)
    validation_losses.append(val_loss)

    if epoch % 10 == 0:
        print(f'Epoch {epoch}, Train Loss: {train_loss:.4f}, Val Loss:
{val_loss:.4f}')

# Plot training and validation losses
plt.figure(figsize=(10, 5))
plt.plot(training_losses, label='Training Loss')
plt.plot(validation_losses, label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training and Validation Loss over Time')
plt.legend()
plt.show()

# Plot decision boundaries with both training and validation data
plt.figure(figsize=(10, 8))
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
```

```
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100),
                     np.linspace(y_min, y_max, 100))

Z = np.zeros(xx.shape)
for i in range(xx.shape[0]):
    for j in range(xx.shape[1]):
        Z[i, j] = perceptron.forward(np.array([xx[i, j], yy[i, j]]))

plt.contourf(xx, yy, Z, levels=[0, 0.5, 1], cmap=plt.cm.RdYlBu, alpha=0.6)

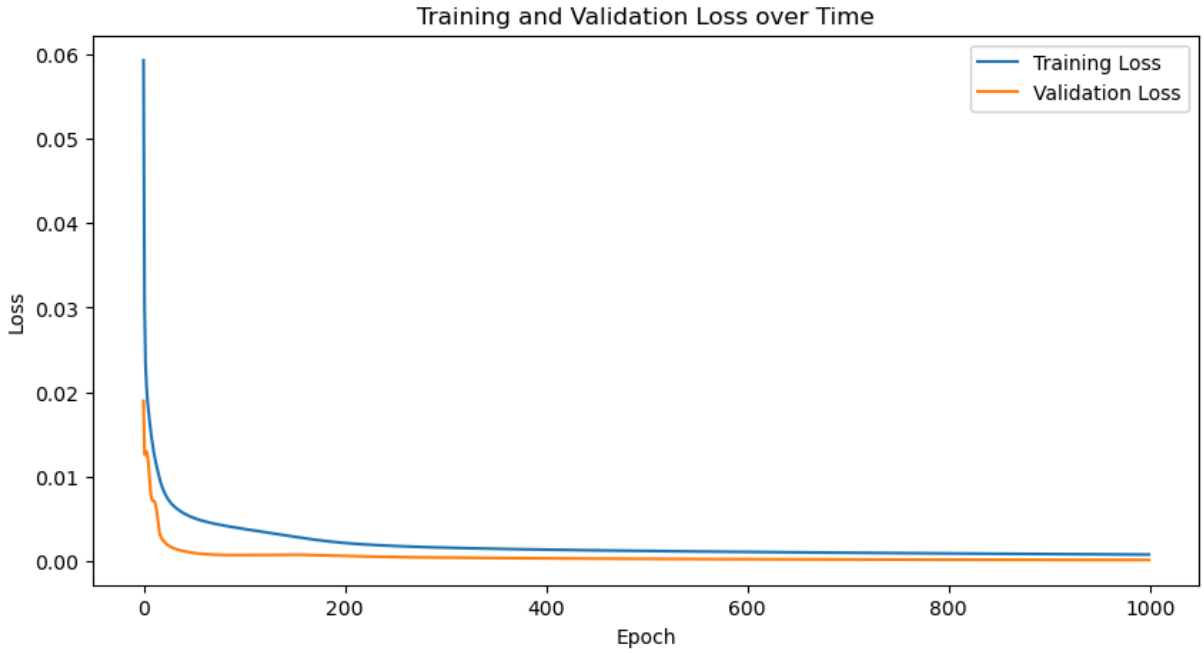
# Plot training data
plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train,
           cmap=plt.cm.RdYlBu, marker='o', label='Training Data')

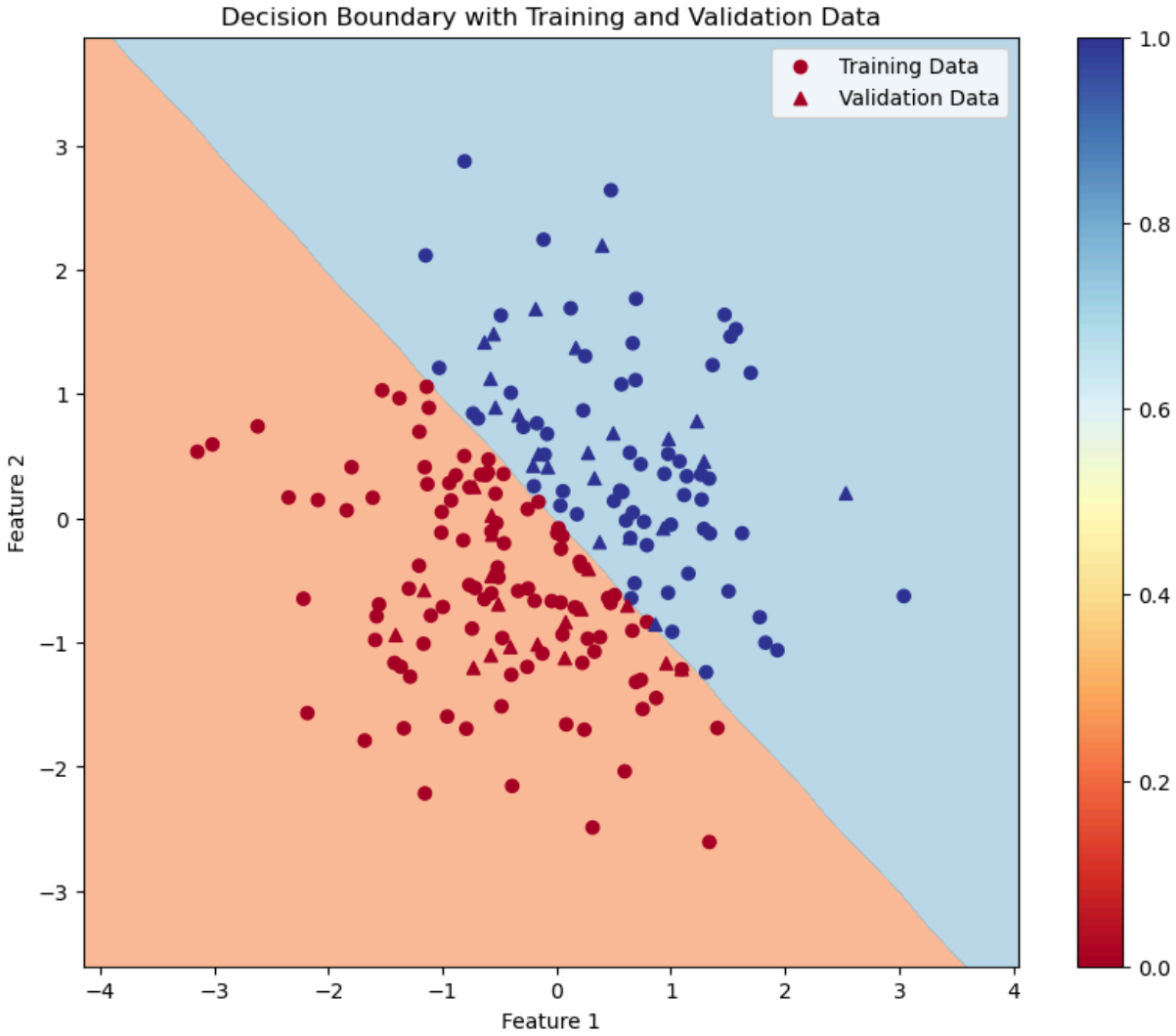
# Plot validation data
plt.scatter(X_val[:, 0], X_val[:, 1], c=y_val,
           cmap=plt.cm.RdYlBu, marker='^', label='Validation Data')

plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Decision Boundary with Training and Validation Data')
plt.legend()
plt.colorbar()
plt.show()
```

```
Epoch 0, Train Loss: 0.0593, Val Loss: 0.0190
Epoch 10, Train Loss: 0.0130, Val Loss: 0.0071
Epoch 20, Train Loss: 0.0083, Val Loss: 0.0023
Epoch 30, Train Loss: 0.0065, Val Loss: 0.0015
Epoch 40, Train Loss: 0.0056, Val Loss: 0.0011
Epoch 50, Train Loss: 0.0051, Val Loss: 0.0009
Epoch 60, Train Loss: 0.0047, Val Loss: 0.0008
Epoch 70, Train Loss: 0.0044, Val Loss: 0.0007
Epoch 80, Train Loss: 0.0042, Val Loss: 0.0007
Epoch 90, Train Loss: 0.0040, Val Loss: 0.0007
Epoch 100, Train Loss: 0.0038, Val Loss: 0.0007
Epoch 110, Train Loss: 0.0036, Val Loss: 0.0007
Epoch 120, Train Loss: 0.0034, Val Loss: 0.0007
Epoch 130, Train Loss: 0.0032, Val Loss: 0.0007
Epoch 140, Train Loss: 0.0030, Val Loss: 0.0007
Epoch 150, Train Loss: 0.0028, Val Loss: 0.0007
Epoch 160, Train Loss: 0.0027, Val Loss: 0.0007
Epoch 170, Train Loss: 0.0025, Val Loss: 0.0007
Epoch 180, Train Loss: 0.0023, Val Loss: 0.0006
Epoch 190, Train Loss: 0.0022, Val Loss: 0.0006
Epoch 200, Train Loss: 0.0021, Val Loss: 0.0006
Epoch 210, Train Loss: 0.0020, Val Loss: 0.0005
Epoch 220, Train Loss: 0.0019, Val Loss: 0.0005
Epoch 230, Train Loss: 0.0019, Val Loss: 0.0005
Epoch 240, Train Loss: 0.0018, Val Loss: 0.0005
Epoch 250, Train Loss: 0.0017, Val Loss: 0.0004
Epoch 260, Train Loss: 0.0017, Val Loss: 0.0004
Epoch 270, Train Loss: 0.0017, Val Loss: 0.0004
Epoch 280, Train Loss: 0.0016, Val Loss: 0.0004
Epoch 290, Train Loss: 0.0016, Val Loss: 0.0004
Epoch 300, Train Loss: 0.0015, Val Loss: 0.0004
Epoch 310, Train Loss: 0.0015, Val Loss: 0.0004
Epoch 320, Train Loss: 0.0015, Val Loss: 0.0003
Epoch 330, Train Loss: 0.0015, Val Loss: 0.0003
Epoch 340, Train Loss: 0.0014, Val Loss: 0.0003
Epoch 350, Train Loss: 0.0014, Val Loss: 0.0003
Epoch 360, Train Loss: 0.0014, Val Loss: 0.0003
Epoch 370, Train Loss: 0.0014, Val Loss: 0.0003
Epoch 380, Train Loss: 0.0013, Val Loss: 0.0003
Epoch 390, Train Loss: 0.0013, Val Loss: 0.0003
Epoch 400, Train Loss: 0.0013, Val Loss: 0.0003
Epoch 410, Train Loss: 0.0013, Val Loss: 0.0003
Epoch 420, Train Loss: 0.0013, Val Loss: 0.0003
Epoch 430, Train Loss: 0.0013, Val Loss: 0.0003
Epoch 440, Train Loss: 0.0012, Val Loss: 0.0002
Epoch 450, Train Loss: 0.0012, Val Loss: 0.0002
Epoch 460, Train Loss: 0.0012, Val Loss: 0.0002
Epoch 470, Train Loss: 0.0012, Val Loss: 0.0002
Epoch 480, Train Loss: 0.0012, Val Loss: 0.0002
Epoch 490, Train Loss: 0.0012, Val Loss: 0.0002
Epoch 500, Train Loss: 0.0012, Val Loss: 0.0002
Epoch 510, Train Loss: 0.0011, Val Loss: 0.0002
Epoch 520, Train Loss: 0.0011, Val Loss: 0.0002
Epoch 530, Train Loss: 0.0011, Val Loss: 0.0002
Epoch 540, Train Loss: 0.0011, Val Loss: 0.0002
Epoch 550, Train Loss: 0.0011, Val Loss: 0.0002
```

```
Epoch 560, Train Loss: 0.0011, Val Loss: 0.0002
Epoch 570, Train Loss: 0.0011, Val Loss: 0.0002
Epoch 580, Train Loss: 0.0011, Val Loss: 0.0002
Epoch 590, Train Loss: 0.0010, Val Loss: 0.0002
Epoch 600, Train Loss: 0.0010, Val Loss: 0.0002
Epoch 610, Train Loss: 0.0010, Val Loss: 0.0002
Epoch 620, Train Loss: 0.0010, Val Loss: 0.0002
Epoch 630, Train Loss: 0.0010, Val Loss: 0.0002
Epoch 640, Train Loss: 0.0010, Val Loss: 0.0002
Epoch 650, Train Loss: 0.0010, Val Loss: 0.0002
Epoch 660, Train Loss: 0.0010, Val Loss: 0.0002
Epoch 670, Train Loss: 0.0010, Val Loss: 0.0002
Epoch 680, Train Loss: 0.0010, Val Loss: 0.0002
Epoch 690, Train Loss: 0.0010, Val Loss: 0.0001
Epoch 700, Train Loss: 0.0009, Val Loss: 0.0001
Epoch 710, Train Loss: 0.0009, Val Loss: 0.0001
Epoch 720, Train Loss: 0.0009, Val Loss: 0.0001
Epoch 730, Train Loss: 0.0009, Val Loss: 0.0001
Epoch 740, Train Loss: 0.0009, Val Loss: 0.0001
Epoch 750, Train Loss: 0.0009, Val Loss: 0.0001
Epoch 760, Train Loss: 0.0009, Val Loss: 0.0001
Epoch 770, Train Loss: 0.0009, Val Loss: 0.0001
Epoch 780, Train Loss: 0.0009, Val Loss: 0.0001
Epoch 790, Train Loss: 0.0009, Val Loss: 0.0001
Epoch 800, Train Loss: 0.0009, Val Loss: 0.0001
Epoch 810, Train Loss: 0.0009, Val Loss: 0.0001
Epoch 820, Train Loss: 0.0008, Val Loss: 0.0001
Epoch 830, Train Loss: 0.0008, Val Loss: 0.0001
Epoch 840, Train Loss: 0.0008, Val Loss: 0.0001
Epoch 850, Train Loss: 0.0008, Val Loss: 0.0001
Epoch 860, Train Loss: 0.0008, Val Loss: 0.0001
Epoch 870, Train Loss: 0.0008, Val Loss: 0.0001
Epoch 880, Train Loss: 0.0008, Val Loss: 0.0001
Epoch 890, Train Loss: 0.0008, Val Loss: 0.0001
Epoch 900, Train Loss: 0.0008, Val Loss: 0.0001
Epoch 910, Train Loss: 0.0008, Val Loss: 0.0001
Epoch 920, Train Loss: 0.0008, Val Loss: 0.0001
Epoch 930, Train Loss: 0.0008, Val Loss: 0.0001
Epoch 940, Train Loss: 0.0008, Val Loss: 0.0001
Epoch 950, Train Loss: 0.0008, Val Loss: 0.0001
Epoch 960, Train Loss: 0.0008, Val Loss: 0.0001
Epoch 970, Train Loss: 0.0007, Val Loss: 0.0001
Epoch 980, Train Loss: 0.0007, Val Loss: 0.0001
Epoch 990, Train Loss: 0.0007, Val Loss: 0.0001
```


Training and Validation Loss over Time
Training Loss
Validation Loss
Loss
Epoch
0.06
0.05
0.04
0.03
0.02
0.01
0.00
0
200
400
600
800
1000


Decision Boundary with Training and Validation Data
Training Data
Validation Data
Feature 1
Feature 2
1.0
0.8
0.6
0.4
0.2
0.0

2.2

With random weights as given by $[0.4967, -0.1383, 0.6477]$ and bias of 1.5230, and input $x = [1.5, 2.0, -0.5]$, then the weighted sum is given by,

$$z = 1.6677 \qquad , \qquad z = \vec{w} \cdot \vec{x} + b$$

and the predicted output is

$$y = \sigma(z) = 0.8413$$

2.3 Range of values of the perceptron $[0, 1]$

Question 3 (6 pts)

Use your perceptron and example training above to help answer the following questions.

How many free parameters are in the model? (1 pt)

How is gradient descent used? (2 pt)

Describe what happens when the learning rate and the number of epochs are changed. How do the learning curves behave? (3 pts)

1. 3 Free parameters in the model - two weights and one bias
2. Gradient desnet minimizes the loss by iteratively updating the weights and bias in the direction that reduces error. After each forward pass, the gradients D_w and D_b are computed via backpropagation (using the chain rule through the sigmoid), and the paramters are updated as $w = w + \alpha D_w$ and $b = b + \alpha D_b$ where α is the learnign rate !
3. For the data set constructed, the trainining and validation losses go down systematically with increase in epochs ! We also find that after trainign for 1000 epochs the loss is high for a learning rate of 0.001 but very low for a learnign rate of 10 !

Neural Networks

Perceptrons have limited capabilities, but they become very powerful if we connect many of them together. When we connect multiple perceptrons, we can create a neural network. In a fully connected neural network, every node in one layer of the neural network is connected to the next layer. In this example, we'll build a three layer neural network, one input layer with 1 node, one hidden layer with 8 nodes, and an output layer with one node. We'll use this neural network in a simple regression example to learn a polynomial function.

Key Components

1. **Multiple Layers**
    - The hidden layer with 8 nodes allows the network to learn more complex patterns
    - Each node in hidden layer acts like a mini-perceptron
    - Network combines these nodes' outputs to make final prediction
2. **ReLU Activation**
    - Uses Rectified Linear Unit (ReLU) in hidden layer: f(x) = max(0,x)
    - Helps network learn non-linear patterns
    - This activation is commonly used in modern neural networks. It is fast to compute and reduces vanishing gradient problem.
3. **Forward Propagation**
    - Input → Hidden layer weights (W1) and bias (b1)
    - ReLU activation on hidden layer
    - Hidden → Output weights (W2) and bias (b2)
4. **Backpropagation**
    - Calculates gradients for all weights and biases
    - Uses chain rule to propagate error backwards
    - Updates all parameters to minimize loss

Advantages Over Perceptron

1. **Non-linear Learning**
    - Can approximate any continuous function
    - Not limited to linear decision boundaries
    - Better suited for complex real-world problems
2. **Regression Capability**
    - Can output continuous values
    - Not limited to binary classification
    - More versatile for different types of problems
3. **Feature Learning**
    - Hidden layer learns useful feature representations
    - Combines features in non-linear ways
    - More powerful than raw input features

Trade-offs

- More complex to train than perceptron
- Requires more computational resources
- More hyperparameters to tune
- Risk of overfitting with too many nodes

We've included the code for a basic neural network below. You can use this code for the next questions, but make sure you understand what it does!

In [23]:
```
import numpy as np

class NeuralNetwork:
    def __init__(self, input_size):
        # Initialize weights and biases
        self.input_size = input_size
        self.hidden_size = 8  # 8 nodes in hidden layer
        self.output_size = 1  # 1 node for regression output

        # Initialize weights with random values
        self.W1 = np.random.randn(self.input_size, self.hidden_size) * 0.01
        self.b1 = np.zeros((1, self.hidden_size))
        self.W2 = np.random.randn(self.hidden_size, self.output_size) *
0.01
        self.b2 = np.zeros((1, self.output_size))

    def relu(self, x):
        return np.maximum(0, x)

    def relu_derivative(self, x):
        return np.where(x > 0, 1, 0)

    def forward(self, X):
        # Forward propagation
        self.z1 = np.dot(X, self.W1) + self.b1
        self.a1 = self.relu(self.z1)
        self.z2 = np.dot(self.a1, self.W2) + self.b2
        return self.z2

    def backward(self, X, y, y_pred, learning_rate):
        m = X.shape[0]

        # Backpropagation
        dz2 = (y_pred - y)
        dW2 = (1/m) * np.dot(self.a1.T, dz2)
        db2 = (1/m) * np.sum(dz2, axis=0, keepdims=True)

        da1 = np.dot(dz2, self.W2.T)
        dz1 = da1 * self.relu_derivative(self.z1)
        dW1 = (1/m) * np.dot(X.T, dz1)
        db1 = (1/m) * np.sum(dz1, axis=0, keepdims=True)

        # Update parameters
        self.W2 -= learning_rate * dW2
        self.b2 -= learning_rate * db2
```

```
        self.W1 -= learning_rate * dW1
        self.b1 -= learning_rate * db1
```

In [24]:

```
def plot_neural_network(layer_sizes, weights=None, biases=None):
    """
    Plot the neural network architecture showing nodes and connections with
labels
    Args:
        layer_sizes: List of number of nodes in each layer
        weights: List of weight matrices between layers
        biases: List of bias vectors for each layer
    """
    fig = plt.figure(figsize=(12, 8))
    vertical_distance_between_layers = 6
    horizontal_distance_between_neurons = 2
    neuron_radius = 0.5

    # Compute positions for neurons in each layer
    layer_positions = []
    for n in layer_sizes:
        layer_y = np.linspace(-(n-1)*horizontal_distance_between_neurons/2,
                              (n-1)*horizontal_distance_between_neurons/2,
n)
        layer_positions.append(layer_y)

    # Plot neurons and connections
    for i, layer_y in enumerate(layer_positions):
        layer_x = i * vertical_distance_between_layers

        # Draw nodes
        for j, y in enumerate(layer_y):
            circle = plt.Circle((layer_x, y), radius=neuron_radius,
fill=False)
            plt.gca().add_patch(circle)

            # Add bias value if provided for non-input layers
            if biases is not None and i > 0:
                plt.text(layer_x-0.7, y+0.7, f'b{i}{j}: {biases[i-1]
[j]:.2f}', fontsize=8)

    # Plot connections with weight labels
    if weights is not None:
        for i, (current_layer_y, next_layer_y) in
enumerate(zip(layer_positions[:-1], layer_positions[1:])):
            current_layer_x = i * vertical_distance_between_layers
            next_layer_x = (i + 1) * vertical_distance_between_layers

            for j, start_y in enumerate(current_layer_y):
                for k, end_y in enumerate(next_layer_y):
                    weight = weights[i][j][k]
                    color = 'red' if weight < 0 else 'blue'
                    #alpha = min(abs(weight), 1.0)
                    alpha = 1
                    plt.plot([current_layer_x, next_layer_x], [start_y,
end_y],
                             color=color, alpha=alpha)
```

```
                    # Add weight label at midpoint
                    mid_x = (current_layer_x + next_layer_x) / 2
                    mid_y = (start_y + end_y) / 2
                    plt.text(mid_x, mid_y, f'w{i}{j}{k}: {weight:.2f}',
                             fontsize=8, ha='center', va='center')

    # Add layer labels
    x_positions = np.arange(len(layer_sizes)) *
vertical_distance_between_layers
    plt.xticks(x_positions, [f'Layer {i}\n({size} nodes)'
                             for i, size in enumerate(layer_sizes)])

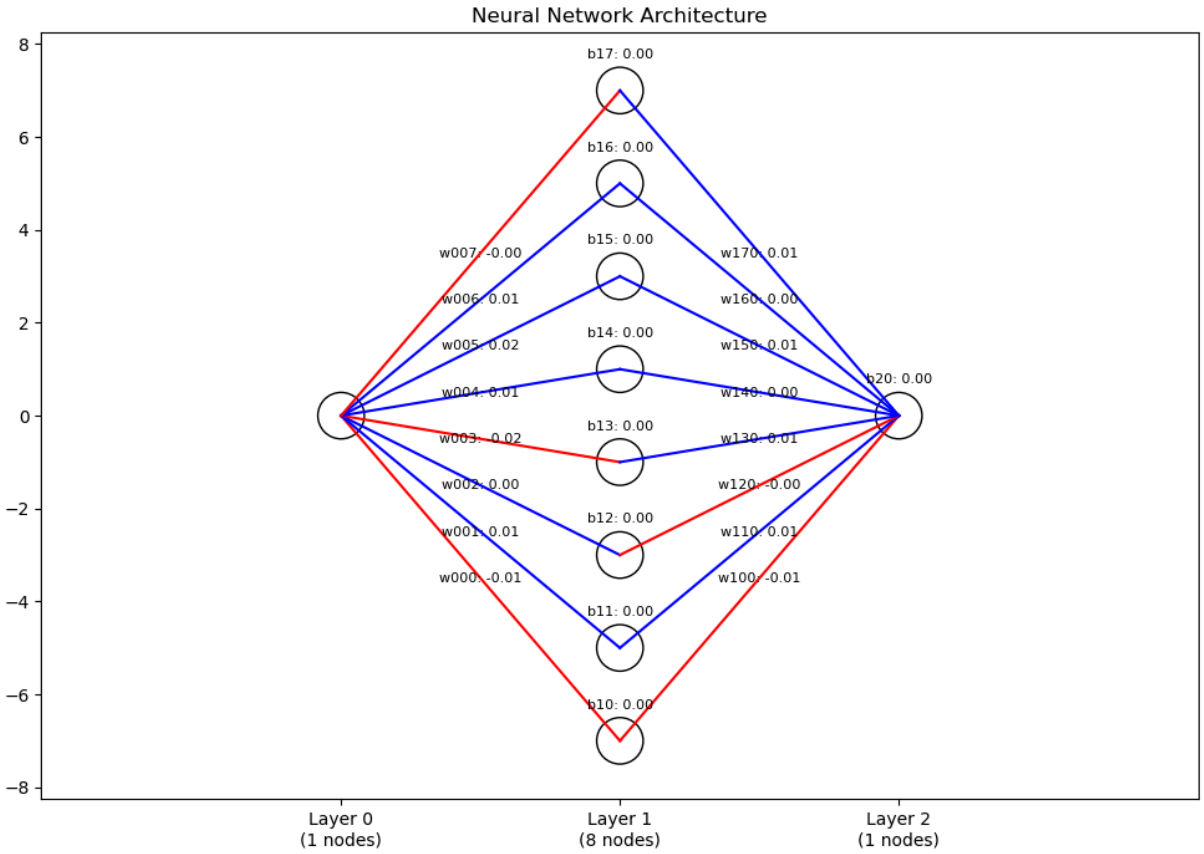
    plt.ylim(-4, 4)
    plt.title('Neural Network Architecture')
    plt.axis('equal')
    plt.show()
```

In [25]:
```
# Generate synthetic data
np.random.seed(0)
X = np.sort(5 * np.random.rand(100, 1), axis=0)
y = X**2+1
#y = np.sin(X).ravel() + np.random.normal(0, 0.1, X.shape[0])
y = y.reshape(-1, 1)

# Initialize neural network
model = NeuralNetwork(input_size=1)
```

In [27]:
```
weights = [model.W1, model.W2]  # Do not transpose weights
biases = [model.b1.flatten(), model.b2.flatten()]  # Flatten to convert to
1D arrays
layer_sizes = [1, 8, 1]  # Input layer, hidden layer, output layer

# Visualize network architecture
plot_neural_network(layer_sizes, weights, biases)
```



In [28]:
```python
# Training parameters
epochs = 1000
learning_rate = 0.01
losses = []

# Train the model
for epoch in range(epochs):
    # Forward pass
    y_pred = model.forward(X)

    # Calculate loss
    loss = np.mean((y_pred - y) ** 2)
    losses.append(loss)

    # Backward pass
    model.backward(X, y, y_pred, learning_rate)

    if epoch % 100 == 0:
        print(f'Epoch {epoch}, Loss: {loss:.4f}')

# Plot results
plt.figure(figsize=(12, 5))

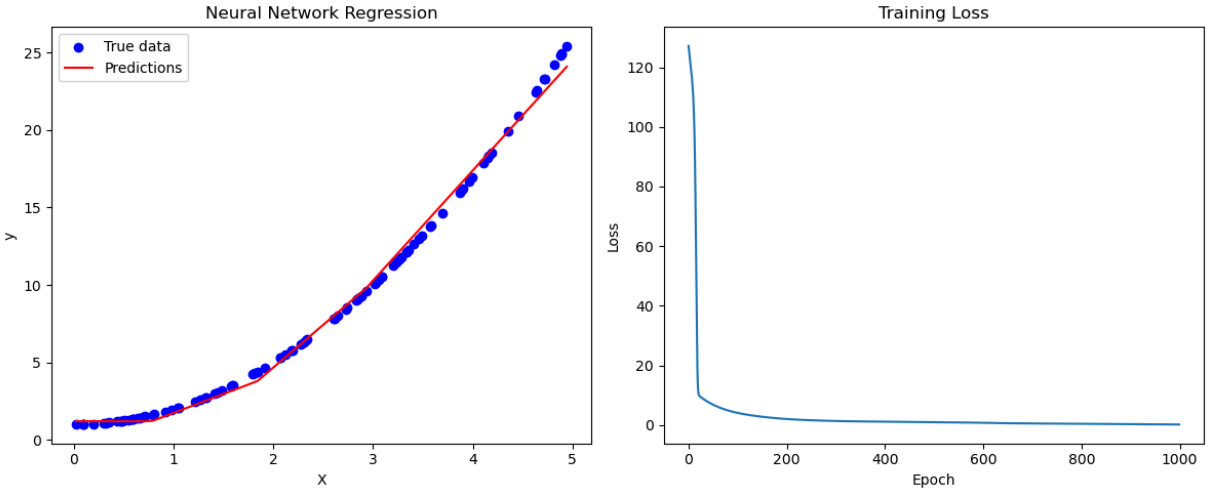
# Plot training results
plt.subplot(1, 2, 1)
plt.scatter(X, y, color='blue', label='True data')
plt.plot(X, model.forward(X), color='red', label='Predictions')
plt.xlabel('X')
plt.ylabel('y')
```

```
plt.title('Neural Network Regression')
plt.legend()

# Plot loss curve
plt.subplot(1, 2, 2)
plt.plot(losses)
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training Loss')

plt.tight_layout()
plt.show()
```

```
Epoch 0, Loss: 127.1119
Epoch 100, Loss: 4.0602
Epoch 200, Loss: 2.0151
Epoch 300, Loss: 1.3507
Epoch 400, Loss: 1.1344
Epoch 500, Loss: 1.0005
Epoch 600, Loss: 0.7408
Epoch 700, Loss: 0.5323
Epoch 800, Loss: 0.4085
Epoch 900, Loss: 0.2734
```



Question 4 (11 pts)

Using the neural network from above, answer the following:

How many free parameters are there? How did you determine this? (1 pt)

How is gradient descent used? (2 pts)

Describe the quality of the fit to the example data with the default neural network. (1 pt)

Hypothesize why it might take so many more model parameters to fit a second order polynomial? (1 pt)

How does the changing the architecture such as the number of nodes in the hidden model affect the quality of the fit? (2 pts)

Increase the order of the polynomial that generates the data and explore how the relationship between the number of nodes and the quality of the fit for different polynomial orders. What would you conclude about the use of neural networks for this task? (4 pts)

1. There are 25 free parameters in the neural network ! There are 8 weights and 8 biases connecting the input with the hidden layer. Then there are 8 weights and 1 bias connecting the hidden layer with the output !

2. GRadient descent principle is the same as before, but now it propagates through two layers instead of one. After the forward pass, the backpropagation computes gradients layer by layer in reverse using the chain rule. Then all the four parameters are updated simultaneously. The key difference from before when we sued the single perceptron is the RELU derivative step. When backpropagating through the hidden layer, the gradiensts are zeroed out for any neuron that was inactive (less than zzero) before, since RELU has zero gradient there.

3. The fit is very good. The red prediction is mostly on top of the true data points across the entire range from 0 to 5, with minor deviations. The training loss confirms this. There is a sharp drop in around the first 100 epochs and converges smoothly by epoch 1000, indicating the networks learned the underlying quadratic relationship very well !

4. A perceptron with sigmoid activation could theoretically fit the data, but it's constrained to a single smooth curve shape. The neural netowrk needs more parameters I think because RELU is piecewise linear and each hidden neuron can only model a straight line segment, so the network must combine multiple neurons to approximate the curve of $x^2 + 1$. The hidden neurons essentially stitch together these linear peices to mimic the curvature which requires more parameters to pull it off !

5. Number of nodes 2 - Performs very badly !!

In [30]:
```
import numpy as np

class NeuralNetwork:
    def __init__(self, input_size):
        # Initialize weights and biases
        self.input_size = input_size
        self.hidden_size = 2 # 2 nodes in hidden layer
        self.output_size = 1  # 1 node for regression output

        # Initialize weights with random values
        self.W1 = np.random.randn(self.input_size, self.hidden_size) * 0.01
        self.b1 = np.zeros((1, self.hidden_size))
```

```
        self.W2 = np.random.randn(self.hidden_size, self.output_size) *
0.01
        self.b2 = np.zeros((1, self.output_size))

    def relu(self, x):
        return np.maximum(0, x)

    def relu_derivative(self, x):
        return np.where(x > 0, 1, 0)

    def forward(self, X):
        # Forward propagation
        self.z1 = np.dot(X, self.W1) + self.b1
        self.a1 = self.relu(self.z1)
        self.z2 = np.dot(self.a1, self.W2) + self.b2
        return self.z2

    def backward(self, X, y, y_pred, learning_rate):
        m = X.shape[0]

        # Backpropagation
        dz2 = (y_pred - y)
        dW2 = (1/m) * np.dot(self.a1.T, dz2)
        db2 = (1/m) * np.sum(dz2, axis=0, keepdims=True)

        da1 = np.dot(dz2, self.W2.T)
        dz1 = da1 * self.relu_derivative(self.z1)
        dW1 = (1/m) * np.dot(X.T, dz1)
        db1 = (1/m) * np.sum(dz1, axis=0, keepdims=True)

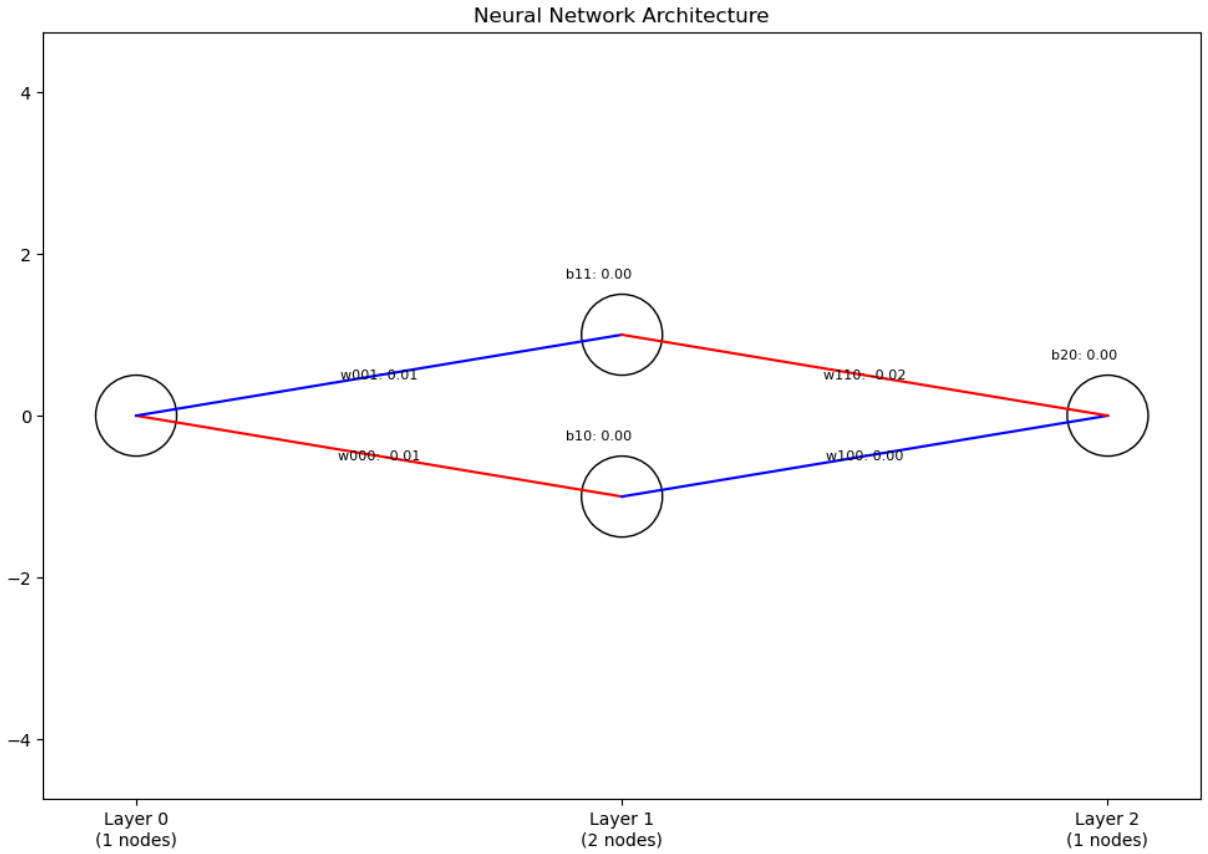
        # Update parameters
        self.W2 -= learning_rate * dW2
        self.b2 -= learning_rate * db2
        self.W1 -= learning_rate * dW1
        self.b1 -= learning_rate * db1
```

In [31]:
```
# Generate synthetic data
np.random.seed(0)
X = np.sort(5 * np.random.rand(100, 1), axis=0)
y = X**2+1
#y = np.sin(X).ravel() + np.random.normal(0, 0.1, X.shape[0])
y = y.reshape(-1, 1)

# Initialize neural network
model = NeuralNetwork(input_size=1)
```

In [34]:
```
weights = [model.W1, model.W2]  # Do not transpose weights
biases = [model.b1.flatten(), model.b2.flatten()]  # Flatten to convert to
1D arrays
layer_sizes = [1, 2, 1]  # Input layer, hidden layer, output layer

# Visualize network architecture
plot_neural_network(layer_sizes, weights, biases)
```



In [36]:
```
# Training parameters
epochs = 1000
learning_rate = 0.01
losses = []

# Train the model
for epoch in range(epochs):
    # Forward pass
    y_pred = model.forward(X)

    # Calculate loss
    loss = np.mean((y_pred - y) ** 2)
    losses.append(loss)

    # Backward pass
    model.backward(X, y, y_pred, learning_rate)

    if epoch % 100 == 0:
        print(f'Epoch {epoch}, Loss: {loss:.4f}')

# Plot results
plt.figure(figsize=(12, 5))

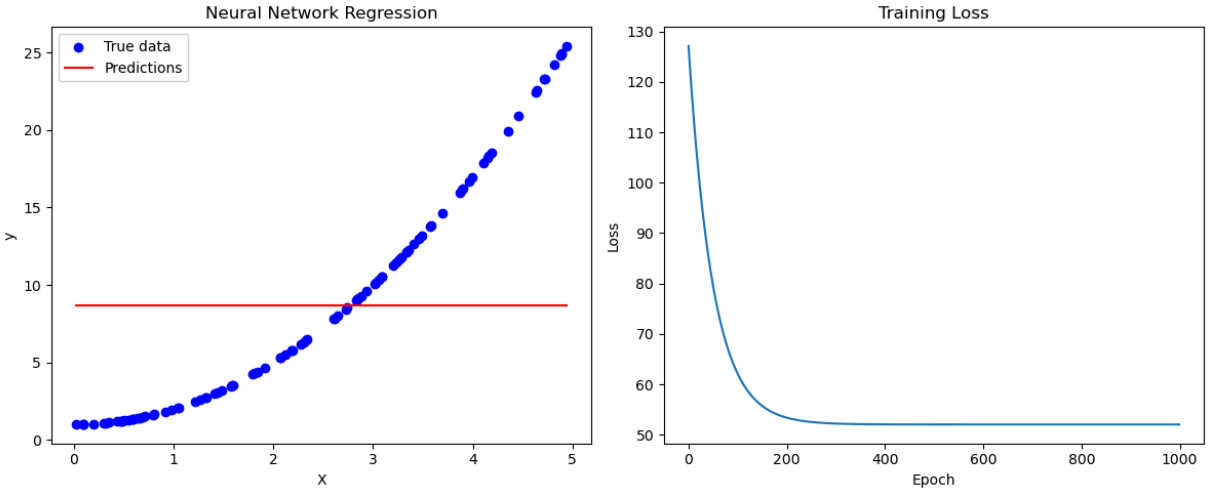
# Plot training results
plt.subplot(1, 2, 1)
plt.scatter(X, y, color='blue', label='True data')
plt.plot(X, model.forward(X), color='red', label='Predictions')
plt.xlabel('X')
plt.ylabel('y')
```

```
plt.title('Neural Network Regression')
plt.legend()

# Plot loss curve
plt.subplot(1, 2, 2)
plt.plot(losses)
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training Loss')

plt.tight_layout()
plt.show()
```

```
Epoch 0, Loss: 127.1407
Epoch 100, Loss: 62.0901
Epoch 200, Loss: 53.3758
Epoch 300, Loss: 52.2082
Epoch 400, Loss: 52.0518
Epoch 500, Loss: 52.0308
Epoch 600, Loss: 52.0280
Epoch 700, Loss: 52.0276
Epoch 800, Loss: 52.0276
Epoch 900, Loss: 52.0276
```



Using too few nodes in the hidden layer 2 is horrible at predicting the quadratic !! This is clear from the graphs above !

5. Number of nodes 64 -- Performs well but this is unnecessary since it does not fare too much better than when we used 8 nodes !

In [37]:
```
import numpy as np

class NeuralNetwork:
    def __init__(self, input_size):
        # Initialize weights and biases
        self.input_size = input_size
        self.hidden_size = 64 # 64 nodes in hidden layer
```

```
        self.output_size = 1  # 1 node for regression output

        # Initialize weights with random values
        self.W1 = np.random.randn(self.input_size, self.hidden_size) * 0.01
        self.b1 = np.zeros((1, self.hidden_size))
        self.W2 = np.random.randn(self.hidden_size, self.output_size) *
0.01
        self.b2 = np.zeros((1, self.output_size))

    def relu(self, x):
        return np.maximum(0, x)

    def relu_derivative(self, x):
        return np.where(x > 0, 1, 0)

    def forward(self, X):
        # Forward propagation
        self.z1 = np.dot(X, self.W1) + self.b1
        self.a1 = self.relu(self.z1)
        self.z2 = np.dot(self.a1, self.W2) + self.b2
        return self.z2

    def backward(self, X, y, y_pred, learning_rate):
        m = X.shape[0]

        # Backpropagation
        dz2 = (y_pred - y)
        dW2 = (1/m) * np.dot(self.a1.T, dz2)
        db2 = (1/m) * np.sum(dz2, axis=0, keepdims=True)

        da1 = np.dot(dz2, self.W2.T)
        dz1 = da1 * self.relu_derivative(self.z1)
        dW1 = (1/m) * np.dot(X.T, dz1)
        db1 = (1/m) * np.sum(dz1, axis=0, keepdims=True)

        # Update parameters
        self.W2 -= learning_rate * dW2
        self.b2 -= learning_rate * db2
        self.W1 -= learning_rate * dW1
        self.b1 -= learning_rate * db1
```

In [38]:
```
# Generate synthetic data
np.random.seed(0)
X = np.sort(5 * np.random.rand(100, 1), axis=0)
y = X**2+1
#y = np.sin(X).ravel() + np.random.normal(0, 0.1, X.shape[0])
y = y.reshape(-1, 1)

# Initialize neural network
model = NeuralNetwork(input_size=1)
```

In [40]:
```
# Training parameters
epochs = 1000
learning_rate = 0.01
losses = []
```

```
# Train the model
for epoch in range(epochs):
    # Forward pass
    y_pred = model.forward(X)

    # Calculate loss
    loss = np.mean((y_pred - y) ** 2)
    losses.append(loss)

    # Backward pass
    model.backward(X, y, y_pred, learning_rate)

    if epoch % 100 == 0:
        print(f'Epoch {epoch}, Loss: {loss:.4f}')

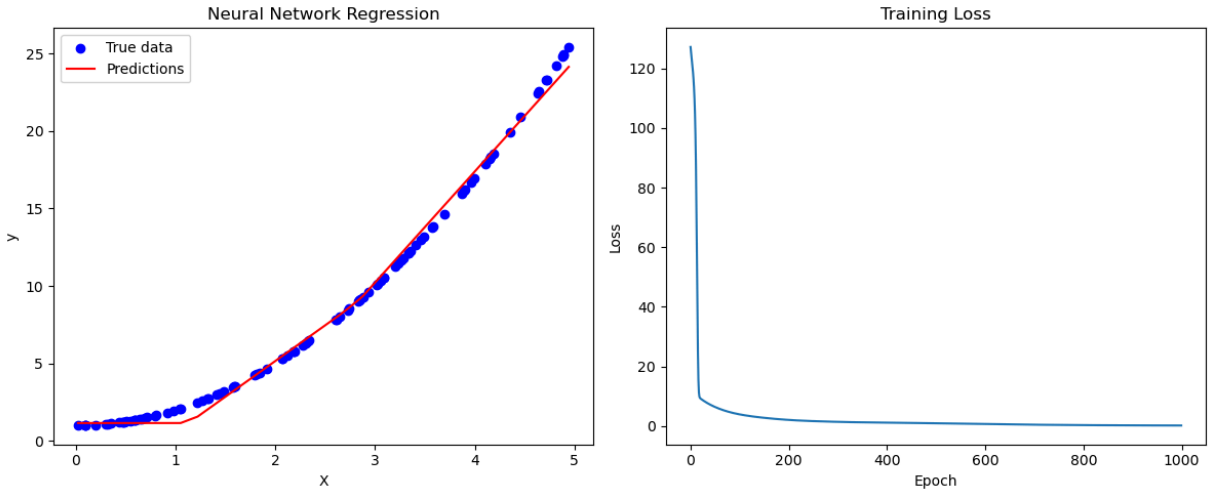
# Plot results
plt.figure(figsize=(12, 5))

# Plot training results
plt.subplot(1, 2, 1)
plt.scatter(X, y, color='blue', label='True data')
plt.plot(X, model.forward(X), color='red', label='Predictions')
plt.xlabel('X')
plt.ylabel('y')
plt.title('Neural Network Regression')
plt.legend()

# Plot loss curve
plt.subplot(1, 2, 2)
plt.plot(losses)
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training Loss')

plt.tight_layout()
plt.show()
```

```
Epoch 0, Loss: 127.1944
Epoch 100, Loss: 3.9342
Epoch 200, Loss: 2.0825
Epoch 300, Loss: 1.4324
Epoch 400, Loss: 1.1757
Epoch 500, Loss: 0.9581
Epoch 600, Loss: 0.6763
Epoch 700, Loss: 0.4756
Epoch 800, Loss: 0.3397
Epoch 900, Loss: 0.2569
```



64 nodes reproduces the quadratic very well, but this is clearly overkill ! The accuracy with which it predicts the quadratic is not too better than when we just used 8 nodes in the hidden layer ! This is clear from the regression graph but also from the training loss behaviour with epochs !

6. Change the data to a higher order polynomial -

$$x^4 + 5x^3 + 6x + 1$$

Start with 8 nodes

In [57]:
```
import numpy as np

class NeuralNetwork:
    def __init__(self, input_size):
        # Initialize weights and biases
        self.input_size = input_size
        self.hidden_size = 8 # 8 nodes in hidden layer
        self.output_size = 1  # 1 node for regression output

        # Initialize weights with random values
        self.W1 = np.random.randn(self.input_size, self.hidden_size) * 0.01
        self.b1 = np.zeros((1, self.hidden_size))
        self.W2 = np.random.randn(self.hidden_size, self.output_size) *
0.01
        self.b2 = np.zeros((1, self.output_size))

    def relu(self, x):
        return np.maximum(0, x)

    def relu_derivative(self, x):
```

```
        return np.where(x > 0, 1, 0)

    def forward(self, X):
        # Forward propagation
        self.z1 = np.dot(X, self.W1) + self.b1
        self.a1 = self.relu(self.z1)
        self.z2 = np.dot(self.a1, self.W2) + self.b2
        return self.z2

    def backward(self, X, y, y_pred, learning_rate):
        m = X.shape[0]

        # Backpropagation
        dz2 = (y_pred - y)
        dW2 = (1/m) * np.dot(self.a1.T, dz2)
        db2 = (1/m) * np.sum(dz2, axis=0, keepdims=True)

        da1 = np.dot(dz2, self.W2.T)
        dz1 = da1 * self.relu_derivative(self.z1)
        dW1 = (1/m) * np.dot(X.T, dz1)
        db1 = (1/m) * np.sum(dz1, axis=0, keepdims=True)

        # Update parameters
        self.W2 -= learning_rate * dW2
        self.b2 -= learning_rate * db2
        self.W1 -= learning_rate * dW1
        self.b1 -= learning_rate * db1
```

In [58]:
```
# Generate synthetic data
np.random.seed(0)
X = np.sort(5 * np.random.rand(100, 1), axis=0)
y = X**4+5*X**3 + 6*X + 1
#y = np.sin(X).ravel() + np.random.normal(0, 0.1, X.shape[0])
y = y.reshape(-1, 1)

# Initialize neural network
model = NeuralNetwork(input_size=1)
```

In [59]:
```
# Training parameters
epochs = 1000
learning_rate = 0.01
losses = []

# Train the model
for epoch in range(epochs):
    # Forward pass
    y_pred = model.forward(X)

    # Calculate loss
    loss = np.mean((y_pred - y) ** 2)
    losses.append(loss)

    # Backward pass
    model.backward(X, y, y_pred, learning_rate)
```

```
    if epoch % 100 == 0:
        print(f'Epoch {epoch}, Loss: {loss:.4f}')

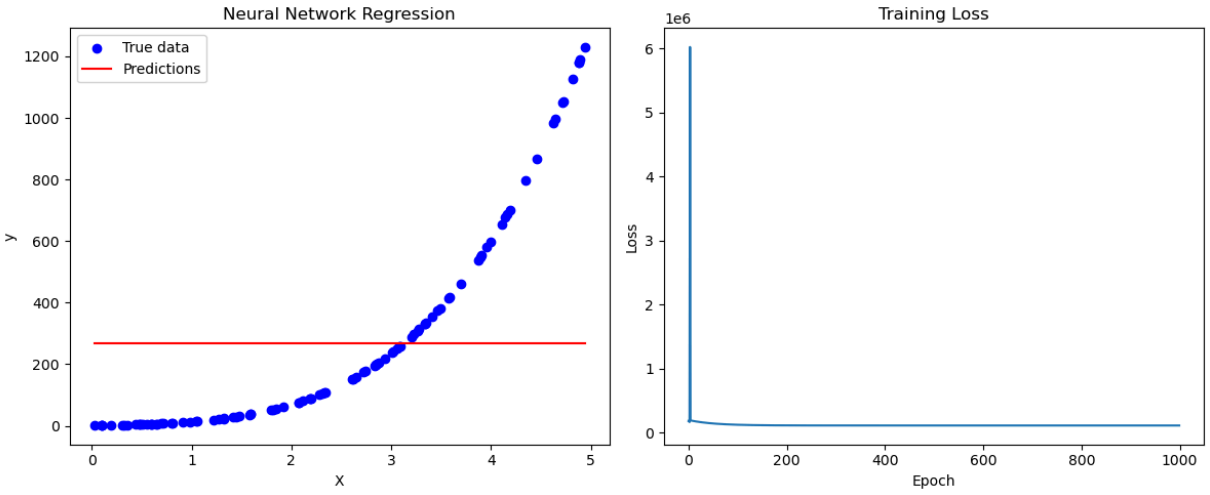
# Plot results
plt.figure(figsize=(12, 5))

# Plot training results
plt.subplot(1, 2, 1)
plt.scatter(X, y, color='blue', label='True data')
plt.plot(X, model.forward(X), color='red', label='Predictions')
plt.xlabel('X')
plt.ylabel('y')
plt.title('Neural Network Regression')
plt.legend()

# Plot loss curve
plt.subplot(1, 2, 2)
plt.plot(losses)
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training Loss')

plt.tight_layout()
plt.show()
```

```
Epoch 0, Loss: 184624.6112
Epoch 100, Loss: 124941.2097
Epoch 200, Loss: 115007.0690
Epoch 300, Loss: 113676.0960
Epoch 400, Loss: 113497.7727
Epoch 500, Loss: 113473.8810
Epoch 600, Loss: 113470.6800
Epoch 700, Loss: 113470.2511
Epoch 800, Loss: 113470.1937
Epoch 900, Loss: 113470.1860
```



Performs extremely bad

Change to 128 nodes !

In [66]:
```
class NeuralNetwork:
    def __init__(self, input_size):
        # Initialize weights and biases
        self.input_size = input_size
        self.hidden_size = 128 # 64 nodes in hidden layer
        self.output_size = 1  # 1 node for regression output

        # Initialize weights with random values
        self.W1 = np.random.randn(self.input_size, self.hidden_size) * 0.01
        self.b1 = np.zeros((1, self.hidden_size))
        self.W2 = np.random.randn(self.hidden_size, self.output_size) *
0.01
        self.b2 = np.zeros((1, self.output_size))

    def relu(self, x):
        return np.maximum(0, x)

    def relu_derivative(self, x):
        return np.where(x > 0, 1, 0)

    def forward(self, X):
        # Forward propagation
        self.z1 = np.dot(X, self.W1) + self.b1
        self.a1 = self.relu(self.z1)
        self.z2 = np.dot(self.a1, self.W2) + self.b2
        return self.z2

    def backward(self, X, y, y_pred, learning_rate):
        m = X.shape[0]

        # Backpropagation
        dz2 = (y_pred - y)
        dW2 = (1/m) * np.dot(self.a1.T, dz2)
        db2 = (1/m) * np.sum(dz2, axis=0, keepdims=True)

        da1 = np.dot(dz2, self.W2.T)
        dz1 = da1 * self.relu_derivative(self.z1)
        dW1 = (1/m) * np.dot(X.T, dz1)
        db1 = (1/m) * np.sum(dz1, axis=0, keepdims=True)

        # Update parameters
        self.W2 -= learning_rate * dW2
        self.b2 -= learning_rate * db2
        self.W1 -= learning_rate * dW1
        self.b1 -= learning_rate * db1

# Generate synthetic data
np.random.seed(0)
X = np.sort(5 * np.random.rand(100, 1), axis=0)
y = X**4+5*X**3 + 6*X + 1
#y = np.sin(X).ravel() + np.random.normal(0, 0.1, X.shape[0])
y = y.reshape(-1, 1)

# Initialize neural network
model = NeuralNetwork(input_size=1)
```

```
# Training parameters
epochs = 10000
learning_rate = 0.0005
losses = []

# Train the model
for epoch in range(epochs):
    # Forward pass
    y_pred = model.forward(X)

    # Calculate loss
    loss = np.mean((y_pred - y) ** 2)
    losses.append(loss)

    # Backward pass
    model.backward(X, y, y_pred, learning_rate)

    if epoch % 100 == 0:
        print(f'Epoch {epoch}, Loss: {loss:.4f}')

# Plot results
plt.figure(figsize=(12, 5))

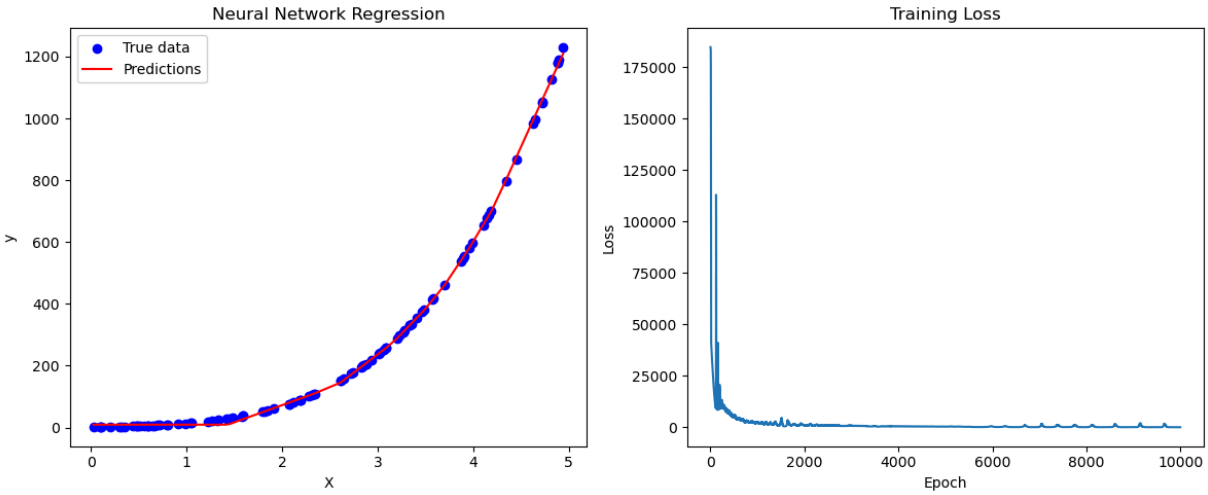
# Plot training results
plt.subplot(1, 2, 1)
plt.scatter(X, y, color='blue', label='True data')
plt.plot(X, model.forward(X), color='red', label='Predictions')
plt.xlabel('X')
plt.ylabel('y')
plt.title('Neural Network Regression')
plt.legend()

# Plot loss curve
plt.subplot(1, 2, 2)
plt.plot(losses)
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Training Loss')

plt.tight_layout()
plt.show()
```

```
Epoch 0, Loss: 184623.9453
Epoch 100, Loss: 11145.9879
Epoch 200, Loss: 17216.1592
Epoch 300, Loss: 10104.7511
Epoch 400, Loss: 7855.8738
Epoch 500, Loss: 5763.3718
Epoch 600, Loss: 4512.6306
Epoch 700, Loss: 3796.0877
Epoch 800, Loss: 2927.0393
Epoch 900, Loss: 2817.7424
Epoch 1000, Loss: 2662.9553
Epoch 1100, Loss: 2653.4965
Epoch 1200, Loss: 2136.7959
Epoch 1300, Loss: 1548.4602
Epoch 1400, Loss: 1744.1288
Epoch 1500, Loss: 3425.7683
Epoch 1600, Loss: 479.9054
Epoch 1700, Loss: 1128.2867
Epoch 1800, Loss: 1354.8908
Epoch 1900, Loss: 1282.7199
Epoch 2000, Loss: 1055.8607
Epoch 2100, Loss: 1196.2544
Epoch 2200, Loss: 720.1503
Epoch 2300, Loss: 833.4078
Epoch 2400, Loss: 793.0884
Epoch 2500, Loss: 812.6259
Epoch 2600, Loss: 788.3373
Epoch 2700, Loss: 799.8478
Epoch 2800, Loss: 693.0746
Epoch 2900, Loss: 705.0824
Epoch 3000, Loss: 777.5880
Epoch 3100, Loss: 720.9491
Epoch 3200, Loss: 692.1859
Epoch 3300, Loss: 664.4398
Epoch 3400, Loss: 421.7809
Epoch 3500, Loss: 668.5484
Epoch 3600, Loss: 367.5507
Epoch 3700, Loss: 463.4281
Epoch 3800, Loss: 557.0061
Epoch 3900, Loss: 507.8552
Epoch 4000, Loss: 500.0622
Epoch 4100, Loss: 470.3725
Epoch 4200, Loss: 452.6332
Epoch 4300, Loss: 432.6169
Epoch 4400, Loss: 419.1925
Epoch 4500, Loss: 407.8749
Epoch 4600, Loss: 393.7096
Epoch 4700, Loss: 407.0405
Epoch 4800, Loss: 346.5059
Epoch 4900, Loss: 354.4480
Epoch 5000, Loss: 335.5618
Epoch 5100, Loss: 393.2876
Epoch 5200, Loss: 306.7511
Epoch 5300, Loss: 366.3445
Epoch 5400, Loss: 245.4512
Epoch 5500, Loss: 250.7569
```

```
Epoch 5600, Loss: 130.4567
Epoch 5700, Loss: 123.1004
Epoch 5800, Loss: 143.5429
Epoch 5900, Loss: 214.5873
Epoch 6000, Loss: 433.9887
Epoch 6100, Loss: 131.8252
Epoch 6200, Loss: 202.5649
Epoch 6300, Loss: 422.8687
Epoch 6400, Loss: 113.1618
Epoch 6500, Loss: 94.6791
Epoch 6600, Loss: 127.7304
Epoch 6700, Loss: 889.1599
Epoch 6800, Loss: 92.7046
Epoch 6900, Loss: 75.6297
Epoch 7000, Loss: 162.9100
Epoch 7100, Loss: 360.7494
Epoch 7200, Loss: 87.7189
Epoch 7300, Loss: 131.6071
Epoch 7400, Loss: 944.7693
Epoch 7500, Loss: 97.9473
Epoch 7600, Loss: 82.0208
Epoch 7700, Loss: 267.9004
Epoch 7800, Loss: 481.3296
Epoch 7900, Loss: 68.2228
Epoch 8000, Loss: 74.2518
Epoch 8100, Loss: 701.2302
Epoch 8200, Loss: 136.7679
Epoch 8300, Loss: 60.2934
Epoch 8400, Loss: 56.4850
Epoch 8500, Loss: 62.4665
Epoch 8600, Loss: 1304.1042
Epoch 8700, Loss: 102.0043
Epoch 8800, Loss: 54.3921
Epoch 8900, Loss: 52.4260
Epoch 9000, Loss: 51.5596
Epoch 9100, Loss: 185.5524
Epoch 9200, Loss: 260.1550
Epoch 9300, Loss: 52.1444
Epoch 9400, Loss: 49.7777
Epoch 9500, Loss: 48.6340
Epoch 9600, Loss: 75.0435
Epoch 9700, Loss: 569.9876
Epoch 9800, Loss: 48.2063
Epoch 9900, Loss: 45.3160
```



- I had to work harder in this case to make the neural network predict the chosen polynomial correctly ! Initially I just increased the number of nodes to 128, but that performed very badly ! What I chose to do next was increase the number of epochs to 10000m but that still did not help the learning process ! As a final change in the hyperparameter space, I lowered the leraning rate to a very small number 0.0005 as opposed to 0.01, as prescirbed in the example. Now I get an architecture which simulates the graph very well ! The training losses are also very low !!

- As the polynomial order increases ,the curve becomes more complex with steeper growth and more curvature. I find that a network with a few hiddesn nodes that fit the quadratic x^2 (8 nodes) well starts to fail on x^4. I require a large number of nodes to maintain a good fit. Higher order polynomials demand more number of linear pieces to approximate teh sharper curves.

- Conclusion : NNs are flexible and can fit increasingly complex functions, but the number of nodes increases with increase in complexity. Higher compelxity requires more parameters (nodes) and more training epochs and becomes more sensitive to the learning rate. The realtionship between model size and fit quality is not straightforward. There is a tradeoff between model capacity, training time and the risk of overfitting (bias-variance tradeoff as discussed in class !).